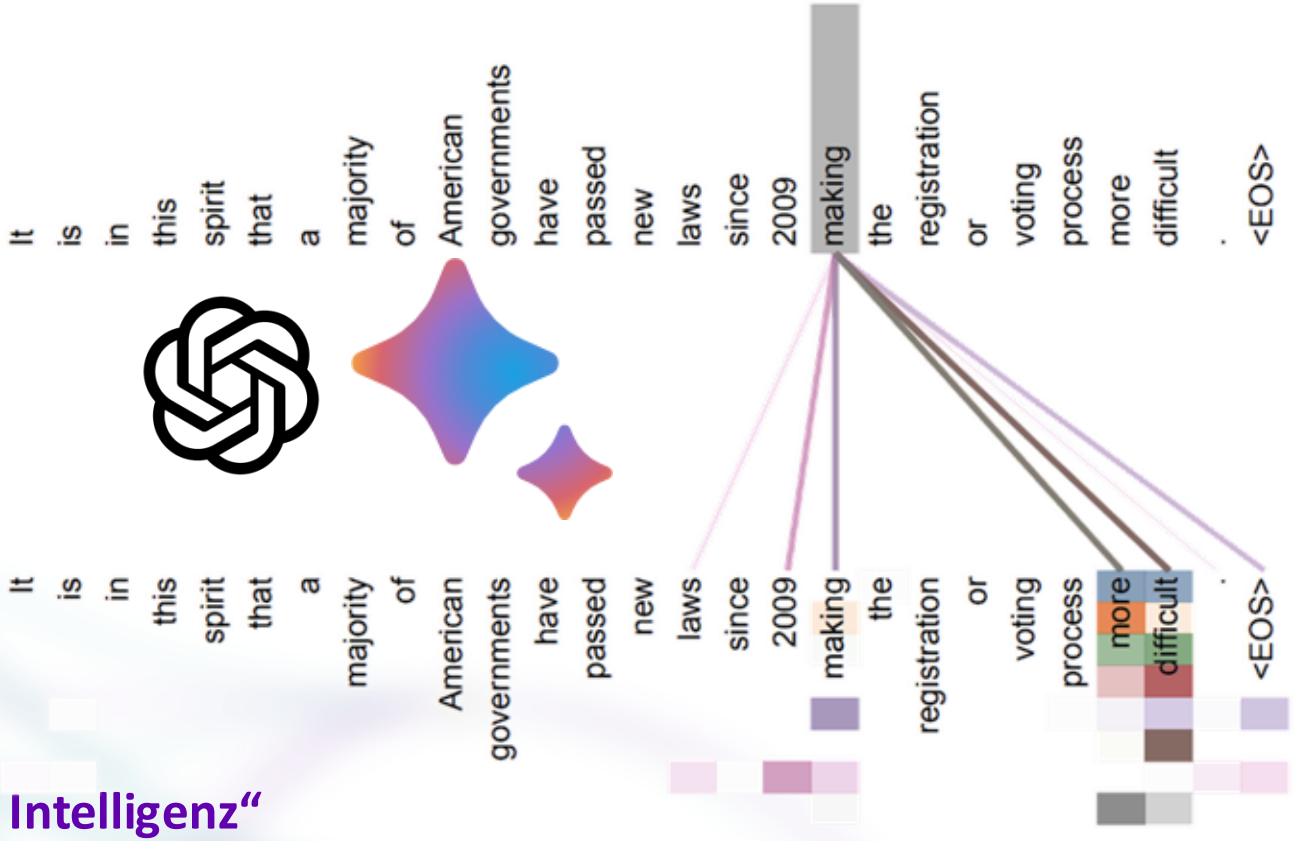
WIE FUNKTIONIERT EIN TRANSFORMER? | iludis.de

Thomas Jörg · Dozent · iludis.de

- GPT, Gemini, Opus usw. sind Transformer.
- Was ist ein Transformer?
- Wie funktioniert ein Transformer?
- Was leisten Transformer?



„Sprache ist der heilige Gral der künstlichen Intelligenz“

https://www.newworldai.com/microsoft-bets-1-billion-holy-grail-artificial-intelligence/

Grundidee: Wortbedeutung durch Kontext | Wortbezüge

„Alex‘ Welpe Toni ist so aufgedreht, sie ist ständig am Wuseln.“

- Die Bedeutung eines Wortes „a“ wird durch das Vorhandensein eines anderen Wortes „b“ erweitert.

Daher "Attention"!

- Im englischen lautet dies „a attends to b“:
  „Das Wort a schenkt dem Wort b Aufmerksamkeit“.



Transformer arbeiten kontextuell | Bezüge von Wörtern zueinander



- Der Kontext eines Textes ergibt sich aus verschiedenen Faktoren, hauptsächlich durch die umgebenden Wörter, die Stilistik usw.
- Der Kontext ist für das Verständnis eines Textes unerlässlich, er ermöglicht es, die Botschaft eines Textes zu erfassen.
- Deshalb arbeitet ein Transformer kontextuell. Titel des Originalartikels:

„Attention is all you need!"

Pointierter: Ohne Residualverbindungen und Feed-Forward-Netze verliert reine Attention mit wachsender Tiefe ihre Ausdruckskraft (Dong et al. 2021) → Kap. 4.

WIE FUNKTIONIERT EIN TRANSFORMER? | iludis.de

Transformer 2017 | „Attention is all you need“

Ursprünglich für maschinelle Übersetzung:

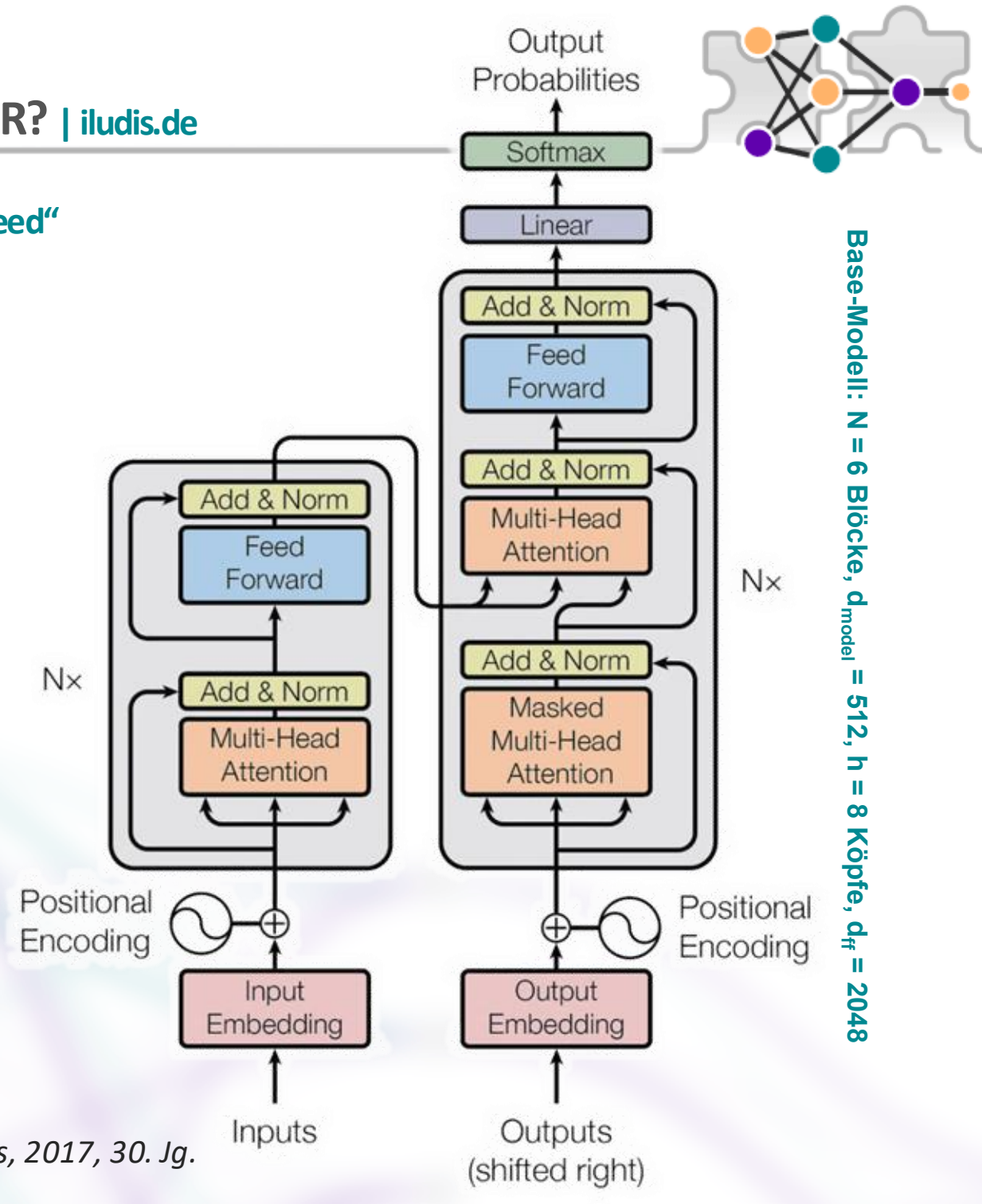
- **Encoder: liest den Quelltext**
- **Decoder: erzeugt den Zieltext Token für Token**

Jeder Block enthält:

- **1. Multi-Head-Attention, im Decoder maskiert,**
- **Cross-Attention**
- **2. Feed-Forward-Netz (FFN)**
- **3. Residualverbindung + LayerNorm**

Davor:

- **Token-Embedding +**
- **Positional Encoding**
- **Linear + Softmax**

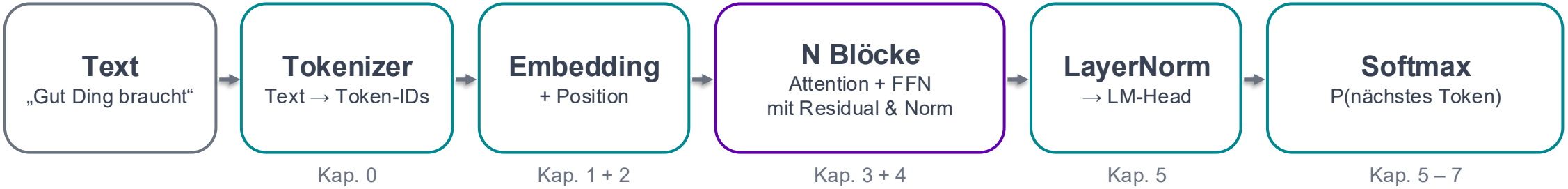


Base-Modell: N = 6 Blöcke, d_{model} = 512, h = 8 Köpfe, d_{ff} = 2048

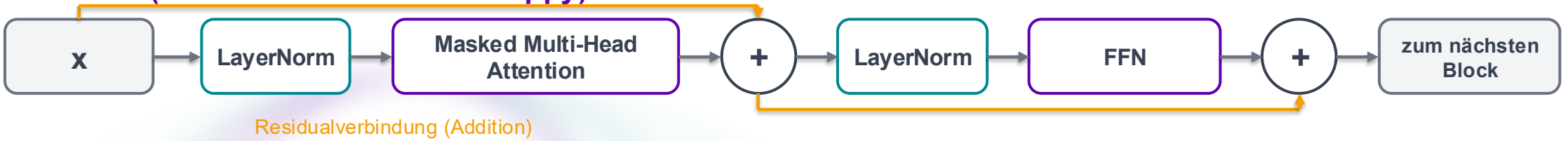
VASWANI, Ashish, et al. Attention is all you need. Advances in neural information processing systems, 2017, 30. Jg.



Überblick | Ein Decoder-only-Transformer (GPT, GuppyLM)



Ein Block (Pre-LN wie GPT-2 und Guppy):



	Vokabular V	d_model	Blöcke N	Köpfe h	d_ff	Kontext	Parameter
GuppyLM (Lehrnotebook)	2 597	384	6	6	1 536	128	11,7 Mio.
GPT-3 175B	–	12 288	96	96	49 152	2 048	175 Mrd.

Guppy: Ausgaben des Lehrnotebooks. GPT-3: Brown et al. (2020), Tab. 2.1 und Abschn. 2.1 (d_ff = 4·d_model, n_ctx = 2048).



0. Tokenisierung | Das Modell sieht Tokens, nicht Wörter

- **Byte Pair Encoding (BPE): Start mit einzelnen Bytes/Zeichen; das häufigste benachbarte Paar wird immer wieder zu einem neuen Token verschmolzen, bis die Vokabulargröße V erreicht ist (*Sennrich et al. 2016*).**
- **Häufige Wörter → ein Token, seltene → mehrere Teilstücke.**
- **Jedes Token hat eine ID (0 … V−1). Mit diesen IDs rechnet das Modell.**
- ***Vokabular: GuppyLM 2 597 · GPT-2 50 257 · Original-Transformer (EN-DE) ≈ 37 000***

Guppy wurde nur auf englischen Dialogen trainiert:
Deutsch zerfällt in viele kleine Stücke.
„Kodierbar“ heißt deshalb nicht automatisch „verstanden“.

Text	Tokens (GuppyLM-BPE)	Anzahl
hello guppy	hello | ␣guppy	2
guppy likes bubbles	guppy | ␣li | k | es | ␣bubbles	5
Gut Ding braucht Weile	G | ut | ␣ | D | ing | ␣b | ra | u | ch | t | ␣ | W | e | i | le	15

Vereinfachung: Auf den folgenden Folien steht oft „Wort“ – gemeint ist immer ein Token.

Tokens ermittelt mit Data/tokenizer.json des Lehrnotebooks; ␣ = Leerzeichen (im Tokenizer als Ġ dargestellt). GPT-2: Radford et al. (2019). Original: Vaswani et al. (2017), Abschn. 5.1.

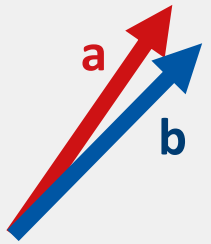
Grundlage: Skalarprodukt | Länge und Richtung

https://iludis.de/vektorMult/index.html

$$\mathbf{a} \cdot \mathbf{b} = \sum_i a_i \, b_i = |\mathbf{a}| \, |\mathbf{b}| \cos\theta$$

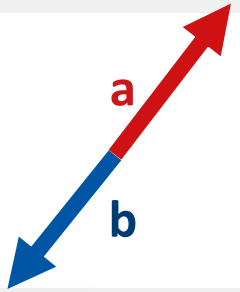
Das Skalarprodukt misst Länge und Richtung zugleich:

- **Gleiche Richtung (cos θ = 1):**
  bei fester Länge maximal positiv.



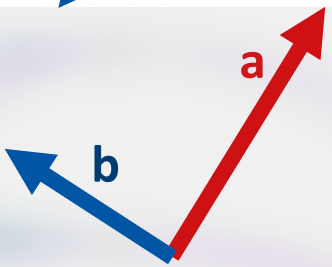
a · b = +max

- **Entgegengesetzt (cos θ = −1):**
  bei fester Länge maximal negativ.



a · b = -max

- **Rechtwinklig (cos θ = 0):**
  Skalarprodukt null.



a · b = 0

Reines Richtungsmaß, Länge herausgerechnet: Kosinusähnlichkeit

$$\cos\theta = \frac{\mathbf{a} \cdot \mathbf{b}}{|\mathbf{a}| \, |\mathbf{b}|}$$

1. “You shall know a word by the company it keeps” | J.R. Firth

Firth (1957): A synopsis of linguistic theory 1930–1955. Oxford: Philological Society.



Ein Text im etablierten Sprachgebrauch enthält Sätze wie:

"Sei nicht so ein Esel!" , "Du dummer Esel!" , "Was für ein Esel er ist!"

In diesen Sätzen findet sich das Wort „Esel“ in vertrauter Gesellschaft.

Du sollst ein Wort an der Gesellschaft erkennen, in der es sich befindet!

1. “You shall know a word by the company it keeps” | J.R. Firth

- **Die Verteilungshypothese**, („distributional hypothesis“)
  ist ein Grundprinzip der Linguistik.

- Sie besagt, dass:
  Wörter, die in ähnlichen Kontexten auftreten,
  tendenziell ähnliche Bedeutungen haben.



Du sollst ein Wort an der Gesellschaft erkennen, in der es sich befindet!

Formuliert u. a. von Harris (1954) und Firth (1957).
→ Auch ein GPT lernt aus Kontext: Es sagt jedes Token aus seiner „Gesellschaft“ vorher (Kap. 6).

Harris, Z. S. (1954): Distributional structure. Word 10 (2–3), 146–162. · Firth, J. R. (1957), s. vorige Folie.

1. “You shall know a word by the company it keeps” | J.R. Firth

- „Wir pflücken die _____ frisch aus dem Garten, um süße Marmelade zuzubereiten.“
- „Direkt vom Feld werden die _____ vom Strauch für einen Joghurt-Smoothie verwendet.“
- „Das rote _____ -Dessert ist nicht nur lecker, sondern auch gesund und vitaminreich.“





1. Exkurs: word2vec (2013) | Statische Embeddings – vor dem Transformer

https://iludis.de/?p=1659

- Skip-Gram: Aus einem Text werden Paare (Wort, Nachbarwort) im Fenster gebildet; ein flaches Netz lernt, die Nachbarn vorherzusagen (Mikolov et al. 2013).
- Ergebnis: ein fester Vektor pro Wort („statische Word-Embeddings“), trainiert außerhalb jedes Transformers.

Wir pflücken die Erdbeeren frisch aus dem Garten

Wir pflücken die Erdbeeren frisch aus dem Garten

Wir pflücken die Erdbeeren frisch aus dem Garten

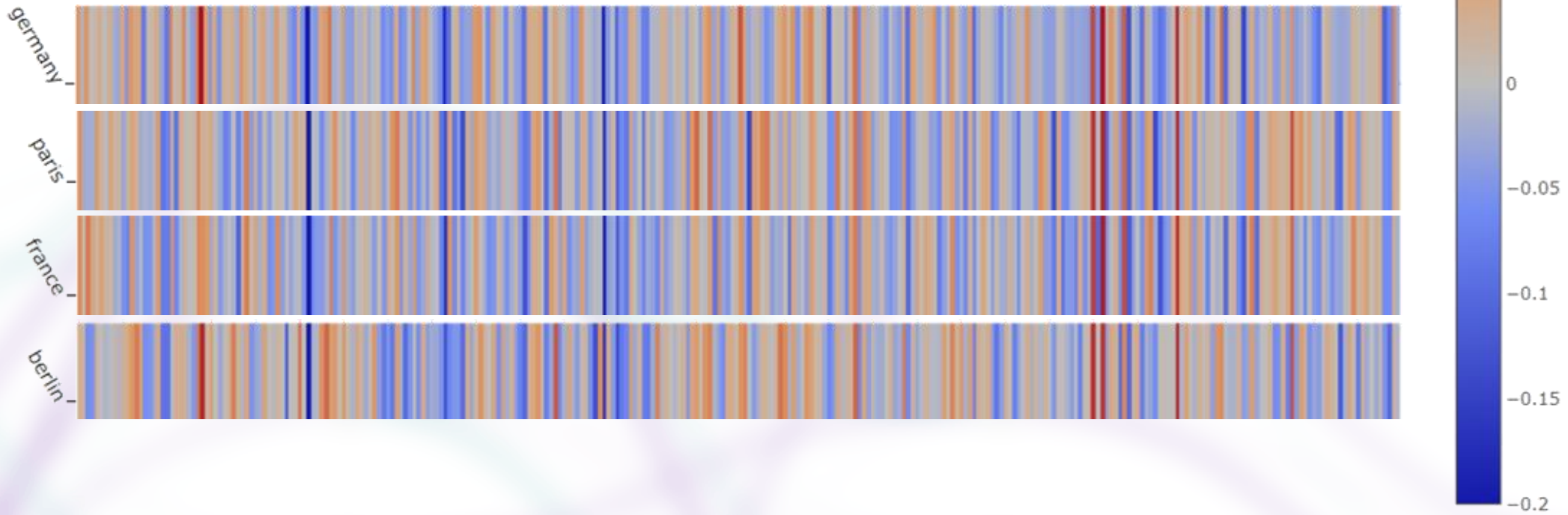
Wir pflücken die Erdbeeren frisch aus dem Garten



1. Exkurs: word2vec | Ergebnis des Trainings: statische Vektoren

https://www.cs.cmu.edu/~dst/WordEmbeddingDemo/index.html
https://iludis.de/WordVektor/index.html

- Die resultierenden Vektoren haben die Form von Listen mit Zahlen.
- Die Listenlänge ist ein frei wählbarer Hyperparameter – hier 300.
- Zum Vergleich, Breite eines Transformers (d_{model}): GPT-3 175B 12 288, Guppy 384.





1. Embeddings im Transformer | Eine gelernte Tabelle – Teil des Modells

- **Jede Token-ID wählt eine Zeile der Matrix W_E ($V \times d_{model}$).**
- **W_E startet zufällig (Guppy: Normalverteilung, $\sigma = 0{,}02$) und wird mit allen anderen Gewichten per Backpropagation auf das Trainingsziel optimiert.**
- **Kein word2vec-Vortraining: „we use learned embeddings“ (Vaswani et al. 2017); GPT-1** *(Radford et al. 2018)*:
  $h_0 = U \cdot W_e + W_p$
- **Guppy: 2 597 × 384 = 997 248 Parameter (8,5 % des Modells).**
- **Weight Tying:**
  Dieselbe Matrix wird am Ausgang wiederverwendet (Kap. 5).

Eingabe des ersten Blocks

$$\mathbf{x}_t^{(0)} = W_E[\mathrm{tok}_t] + \mathbf{p}_t$$

tok_t: Token-ID an Position t; p_t: Positionsvektor (Kap. 2)

$$W_E \in \mathbb{R}^{V \times d_{\mathrm{model}}}$$

Lookup = Multiplikation eines One-hot-Vektors mit W_E: Über den Eingang erhalten nur die Zeilen vorkommender Tokens einen Gradienten; mit Weight Tying kommt über den Ausgang ein Gradient für alle Zeilen hinzu.

Vaswani et al. (2017), Abschn. 3.4 · Radford et al. (2018), Gl. 2 · Press & Wolf (2017) · Guppy: Parameterzählung im Notebook



1. Embeddings im Transformer | Statisch vs. kontextuell

Zeile von W_E (statisch)
ein Vektor pro Token, unabhängig vom Satz
„Bank“ (Sitzbank) = „Bank“ (Geldinstitut)

→

Residualstrom nach Block ℓ (kontextuell)
hängt vom gesamten bisherigen Text ab
„Bank“ im Park ≠ „Bank“ mit Konto

- **Ähnlichkeit per Skalarprodukt ist kein Trainingsziel – Embedding ist nur eingeschränkt interpretierbar.**
- **Kontextuelle Repräsentationen sind stark anisotrop:**
  In der letzten Schicht von GPT-2 haben beliebige Wörter im Mittel eine fast perfekte Kosinusähnlichkeit (*Ethayarajh 2019*).
- **Ein statisches Embedding erklärt weniger als 5 % der Varianz der kontextuellen Vektoren eines Wortes.**
- **In der Attention wird nicht „Ähnlichkeit“ gemessen, sondern eine gelernte Passung $q_i \cdot k_j$ (Kap. 3).**

Ethayarajh (2019): How Contextual are Contextualized Word Representations? EMNLP · Gao et al. (2019): Representation Degeneration Problem, ICLR



2. Positionskodierung | Warum die Stellung im Satz relevant ist

https://iludis.de/positionalEncoding/index.html

„Die, die die Bücher schrieben, sind die, die die Geschichten am besten kennen.“

„Jene“

„welche“

„mehrere“

Gleiche Form „die“, verschiedene Funktion – bestimmt durch Position und Kontext.

Ein weiteres Beispiel:

„Obwohl sie den Preis nicht gewonnen hatte, war sie zufrieden.“

„Obwohl sie den Preis gewonnen hatte, war sie nicht zufrieden.“

2. Positionskodierung | Warum die Stellung im Satz relevant ist

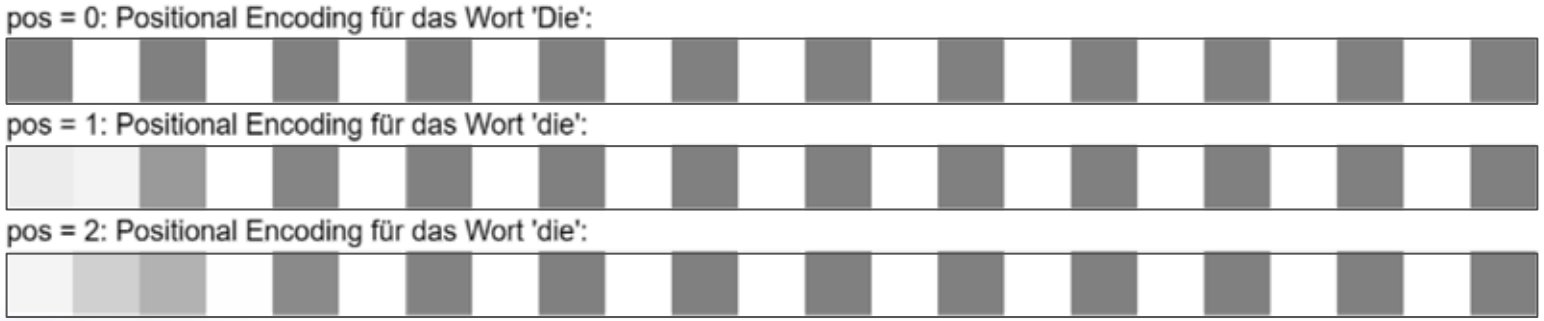
https://iludis.de/positionalEncoding/index.html

„Die, die die Bücher schrieben, sind die, die die Geschichten am besten kennen.“

„Jene“

„welche“

„mehrere“

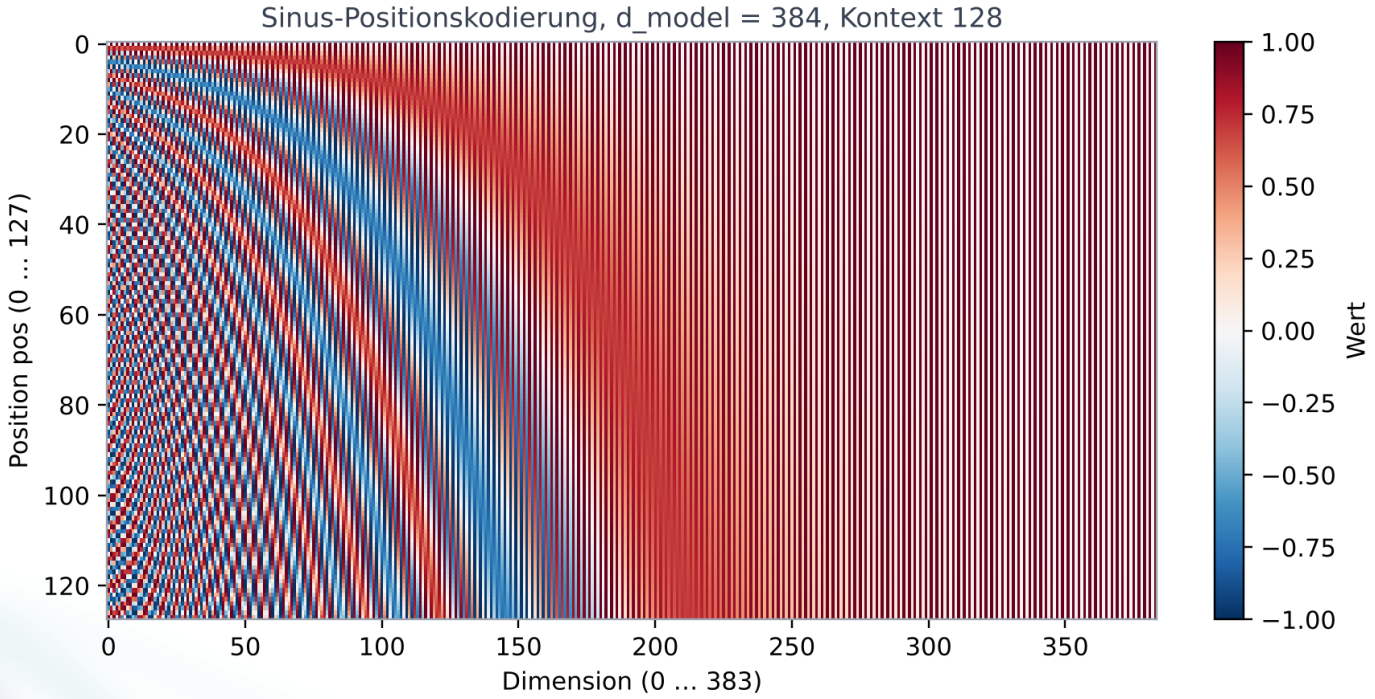


- Self-Attention allein ist reihenfolgeblind:
- Vertauscht man die Eingabe-Tokens, werden die Ausgaben nur mitvertauscht (ohne Maske).
- Deshalb wird zu jedem Token-Embedding ein Positionsvektor addiert.

2. Positionskodierung | Die Sinus-Tabelle (Vaswani et al. 2017)

$$PE_{(pos,\,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{(pos,\,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$



- **Jedes Dimensionspaar schwingt mit eigener Frequenz;**
- **die Wellenlängen reichen geometrisch von 2π bis 10000·2π.**
- **Fest berechnet, nicht trainiert; wird zum Token-Embedding addiert.**
- **Begründung der Autoren: relative Positionen leicht lernbar, evtl. Extrapolation auf längere Folgen.**

Vaswani et al. (2017), Abschn. 3.5: „The wavelengths form a geometric progression from 2π to 10000·2π.“ · Abb.: eigene Darstellung mit Guppy-Maßen



2. Positionskodierung | Gelernt, fest – oder gar nicht?

- Gelernte absolute Positions-Embeddings: eine Tabelle W_P mit einem trainierbaren Vektor je Position – so bei GPT-1 („learned position embeddings instead of the sinusoidal version“), GPT-2 und GPT-3.
- Im Original-Transformer lieferten beide Varianten „nearly identical results“ (Vaswani et al. 2017).
- Absolute Tabellen begrenzen das Kontextfenster: Für Positionen jenseits der Tabelle gibt es keinen Vektor (Guppy: 128 Positionen, GPT-3: 2 048).
- Kausale Sprachmodelle lernen Positionsinformation sogar ganz ohne Positionskodierung – vermutlich, weil die Maske verrät, wie viele Vorgänger ein Token hat (Haviv et al. 2022).
- Heute verbreitet: relative bzw. rotierende Verfahren (z. B. RoPE) – Thema einer späteren Einheit.

Radford et al. (2018) · Vaswani et al. (2017), Abschn. 3.5 und Tab. 3 (E) · Brown et al. (2020) · Haviv et al. (2022): Transformer LMs without Positional Encodings Still Learn Positional Information, Findings of EMNLP

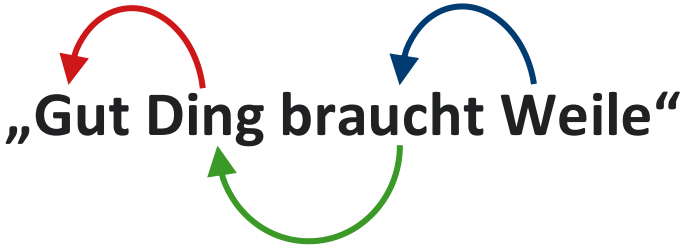
3. Attention | Wortbezüge lassen die Aussage erschließen



In diesem Sprichwort steckt die Aussage, dass

- etwas, das qualitativ gut werden soll, einer längeren Reifezeit bedarf.
- Implizit steckt auch der Gedanke an notwendige Mühe und Fleiß dabei,
- und dass man geduldig abwarten sollte.

3. Attention | Jedes Token wird mit jedem kombiniert



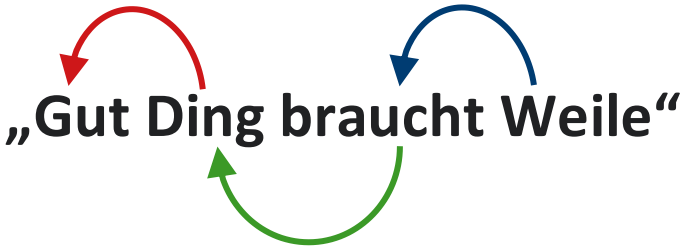
	Gut	*Ding*	*braucht*	*Weile*
Gut	0.4	0.3	0.2	0.1
Ding	**0.6**	0.2	0.1	0.1
braucht	0.2	**0.4**	0.2	0.2
Weile	0.1	0.2	**0.5**	0.2

"Gewicht" nach Softmax: Zeilensumme = 1

- Alle Tokens werden kombiniert,
- Bezüge als Gewichte (ohne Maske).

"Gewicht"

3. Attention | Zeilen = Query, Spalten = Key



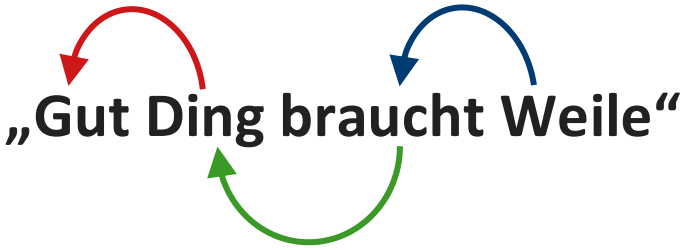
- Zeilen (links beschriftet): Query-Tokens – für sie werden die Gewichte* bestimmt.
- Spalten (oben beschriftet): Key-Tokens – auf sie wird geschaut.

KEY →
QUERY ↓

	Gut	*Ding*	*braucht*	*Weile*
Gut	0.4	0.3	0.2	0.1
Ding	**0.6**	0.2	0.1	0.1
braucht	0.2	**0.4**	0.2	0.2
Weile	0.1	0.2	**0.5**	0.2

*Gewicht = Softmax(Score), Formel folgt.

3. Attention | Kontextvektor eines Tokens



	Gut	*Ding*	*braucht*	*Weile*
Ding	**0.6**	0.2	0.1	0.1

$$Ding_{Kontext} = \mathbf{0.6}\cdot Gut + 0.2\cdot Ding + 0.1\cdot braucht + 0.1\cdot Weile$$

- Eine Zeile ergibt
- durch Addition aller gewichteten Anteile
- den Kontextvektor des Tokens (genauer: Values, s. u.).



3. Attention | Alle Zeilen: Kontextvektoren aller Tokens

$$Gut_{\text{Kontext}} = 0.4 \cdot Gut + 0.3 \cdot Ding + 0.2 \cdot braucht + 0.1 \cdot Weile$$

$$Ding_{\text{Kontext}} = \mathbf{0.6} \cdot Gut + 0.2 \cdot Ding + 0.1 \cdot braucht + 0.1 \cdot Weile$$

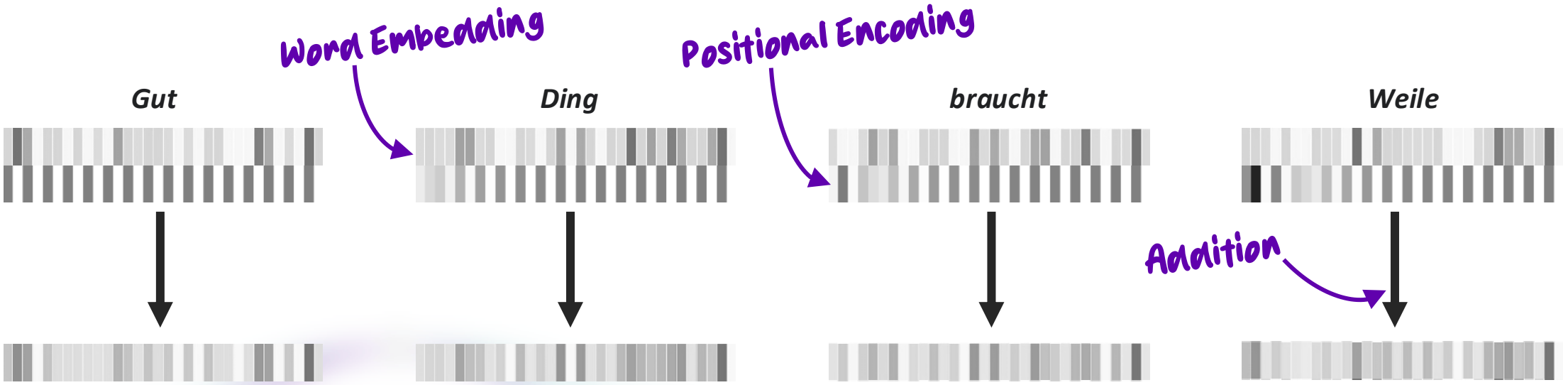
$$braucht_{\text{Kontext}} = 0.2 \cdot Gut + \mathbf{0.4} \cdot Ding + 0.2 \cdot braucht + 0.2 \cdot Weile$$

$$Weile_{\text{Kontext}} = 0.1 \cdot Gut + 0.2 \cdot Ding + \mathbf{0.5} \cdot braucht + 0.2 \cdot Weile$$

- Alle Zeilen zusammen ergeben
- die Kontextvektoren
- aller Tokens des Satzes.

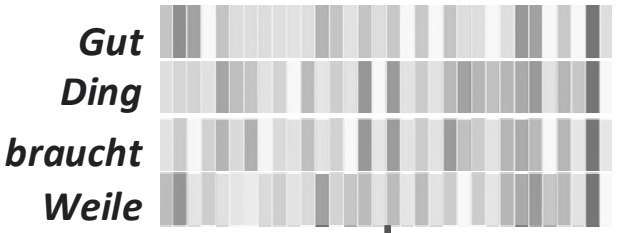
	Gut	*Ding*	*braucht*	*Weile*
Gut	0.4	0.3	0.2	0.1
Ding	**0.6**	0.2	0.1	0.1
braucht	0.2	**0.4**	0.2	0.2
Weile	0.1	0.2	**0.5**	0.2

3. Attention | Schritt 1: Eingabevektoren



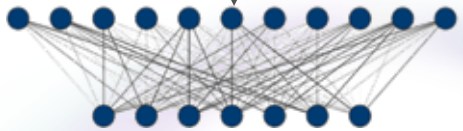
- Alle Tokens werden in ihre Embedding-Vektoren umgewandelt,
- ihr Positionsvektor wird addiert.
- Die Summe ist die Eingabe des ersten Blocks (dort zuerst LayerNorm, dann Attention).

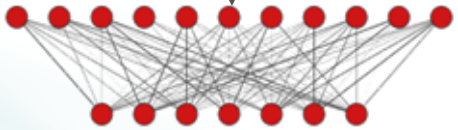
3. Attention | Schritt 2: Query, Key und Value



- Jeder Eingabevektor x wird mit drei gelernten Matrizen multipliziert (linear, ohne Aktivierung).
- Es entstehen dabei die Q- die K- und die V-Vektoren.

$$\mathbf{q} = \mathbf{x}W_Q, \quad \mathbf{k} = \mathbf{x}W_K, \quad \mathbf{v} = \mathbf{x}W_V$$









Query



Key

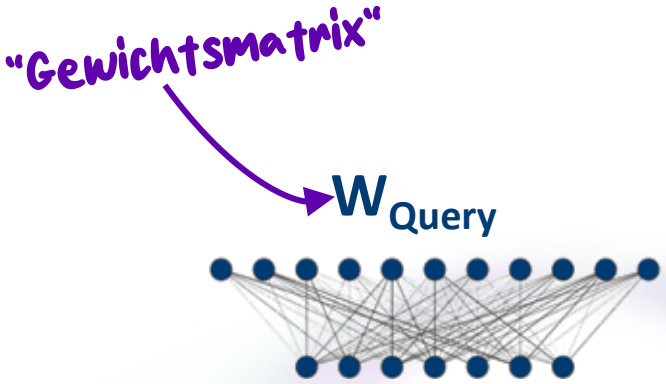


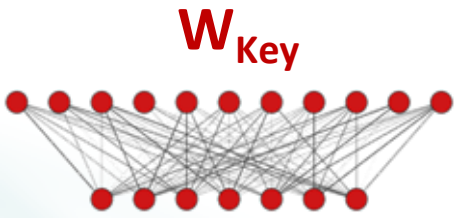
Value

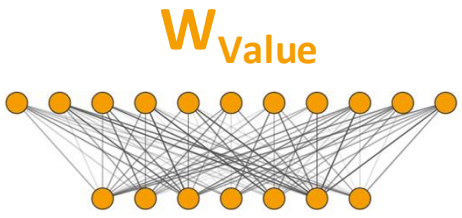


3. Attention | Die Projektionsmatrizen werden gelernt

- W_Q, W_K und W_V sind Parameter des Modells:
- Sie werden im Training angepasst (Kap. 6); jeder Kopf in jedem Block hat eigene.









- Sie lernen, welche Merkmale eines Tokens als Frage (Query), Angebot (Key) und Inhalt (Value) taugen.



3. Attention | Score = Skalarprodukt aus Query und Key

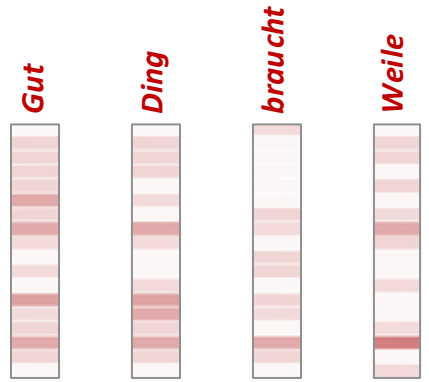
- Score(i, j) = $q_i \cdot k_j$: Wie gut passt das Angebot (Key) von Token j zur Frage (Query) von Token i?
- Gelernt und nicht symmetrisch: Score(i, j) ≠ Score(j, i). Die Softmax macht aus Scores Gewichte (nächste Folie).

	Gut	Ding	braucht	Weile
Gut	0.4	0.3	0.2	0.1
Ding	**0.6**	0.2	0.1	0.1
braucht	0.2	**0.4**	0.2	0.2
Weile	0.1	0.2	**0.5**	0.2



Query

Key





	Gut	Ding	braucht	Weile
Gut	0.4	0.3	0.2	0.1
Ding	**0.6**	0.2	0.1	0.1
braucht	0.2	**0.4**	0.2	0.2
Weile	0.1	0.2	**0.5**	0.2



3. Attention | Von Scores zu Gewichten: die Formel

Score

$$s_{ij} = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$

Gewicht

$$\alpha_{ij} = \frac{\exp(s_{ij})}{\sum_l \exp(s_{il})}$$

Ausgabe

$$\mathbf{z}_i = \sum_j \alpha_{ij}\ \mathbf{v}_j$$

- **Softmax je Zeile: alle Gewichte ≥ 0, Zeilensumme 1.**
- **Warum $\sqrt{d_k}$? Bei zufälligen q, k hat q·k die Varianz d_k. Große Werte machen die Softmax extrem spitz – mit winzigen Gradienten.**
- **Guppy: d_k = 384 / 6 = 64.**
- **M: Maske (nächste Folien).**

In Matrixform:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V$$

Q, K, V: alle Query-, Key-, Value-Vektoren als Zeilen einer Matrix (T × d_k) – so rechnet die Hardware alle Positionen auf einmal.

Vaswani et al. (2017), Abschn. 3.2.1, Fußnote 4: „their dot product, $q \cdot k = \Sigma\, q_i k_i$, has mean 0 and variance d_k“.



3. Attention | Kontextvektor = gewichtete Summe der Values

- **Die Gewichte einer Zeile mischen die Value-Vektoren zum Kontextvektor (hier ohne Maske).**

$$Gut_K = 0.4 \cdot Gut + 0.3 \cdot Ding + 0.2 \cdot braucht + 0.1 \cdot Weile$$

$$Ding_K = 0.6 \cdot Gut + 0.2 \cdot Ding + 0.1 \cdot braucht + 0.1 \cdot Weile$$

$$braucht_K = 0.2 \cdot Gut + 0.4 \cdot Ding + 0.2 \cdot braucht + 0.2 \cdot Weile$$

$$Weile_K = 0.1 \cdot Gut + 0.2 \cdot Ding + 0.5 \cdot braucht + 0.2 \cdot Weile$$

$$Gut_{Kontext} = 0.4 \cdot \text{[Gut]} + 0.3 \cdot \text{[Ding]} + 0.2 \cdot \text{[braucht]} + 0.1 \cdot \text{[Weile]}$$

$$Ding_{Kontext} = 0.6 \cdot \text{[Gut]} + 0.2 \cdot \text{[Ding]} + 0.1 \cdot \text{[braucht]} + 0.1 \cdot \text{[Weile]}$$

$$braucht_{Kontext} = 0.2 \cdot \text{[Gut]} + 0.4 \cdot \text{[Ding]} + 0.2 \cdot \text{[braucht]} + 0.2 \cdot \text{[Weile]}$$

$$Weile_{Kontext} = 0.1 \cdot \text{[Gut]} + 0.2 \cdot \text{[Ding]} + 0.5 \cdot \text{[braucht]} + 0.2 \cdot \text{[Weile]}$$



3. Attention | Kontextvektoren als Farbmuster

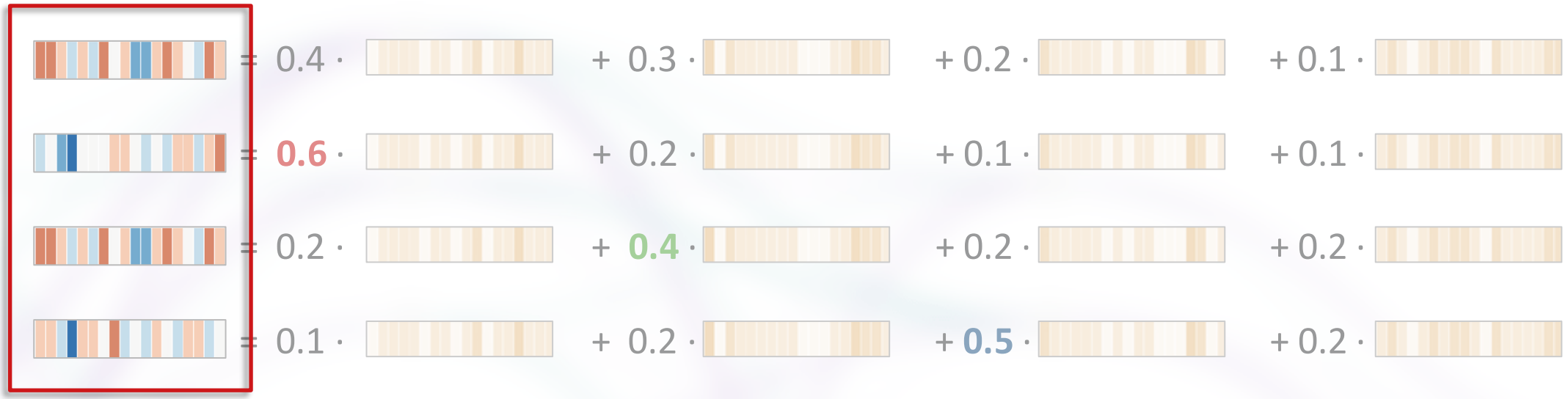
- **Die Gewichte einer Zeile mischen die Value-Vektoren zum Kontextvektor (hier ohne Maske).**

$$\text{Gut}_K = 0.4\cdot\text{Gut} + 0.3\cdot\text{Ding} + 0.2\cdot\text{braucht} + 0.1\cdot\text{Weile}$$

$$\text{Ding}_K = 0.6\cdot\text{Gut} + 0.2\cdot\text{Ding} + 0.1\cdot\text{braucht} + 0.1\cdot\text{Weile}$$

$$\text{braucht}_K = 0.2\cdot\text{Gut} + 0.4\cdot\text{Ding} + 0.2\cdot\text{braucht} + 0.2\cdot\text{Weile}$$

$$\text{Weile}_K = 0.1\cdot\text{Gut} + 0.2\cdot\text{Ding} + 0.5\cdot\text{braucht} + 0.2\cdot\text{Weile}$$





3. Attention | Die kausale Maske: kein Blick in die Zukunft

Maske M (zum Score addiert)

	Gut	Ding	braucht	Weile
Gut	0	$-\infty$	$-\infty$	$-\infty$
Ding	0	0	$-\infty$	$-\infty$
braucht	0	0	0	$-\infty$
Weile	0	0	0	0

Gewichte nach der Softmax

	Gut	Ding	braucht	Weile
Gut	1	0	0	0
Ding	**0.75**	0.25	0	0
braucht	0.25	**0.5**	0.25	0
Weile	0.1	0.2	**0.5**	0.2

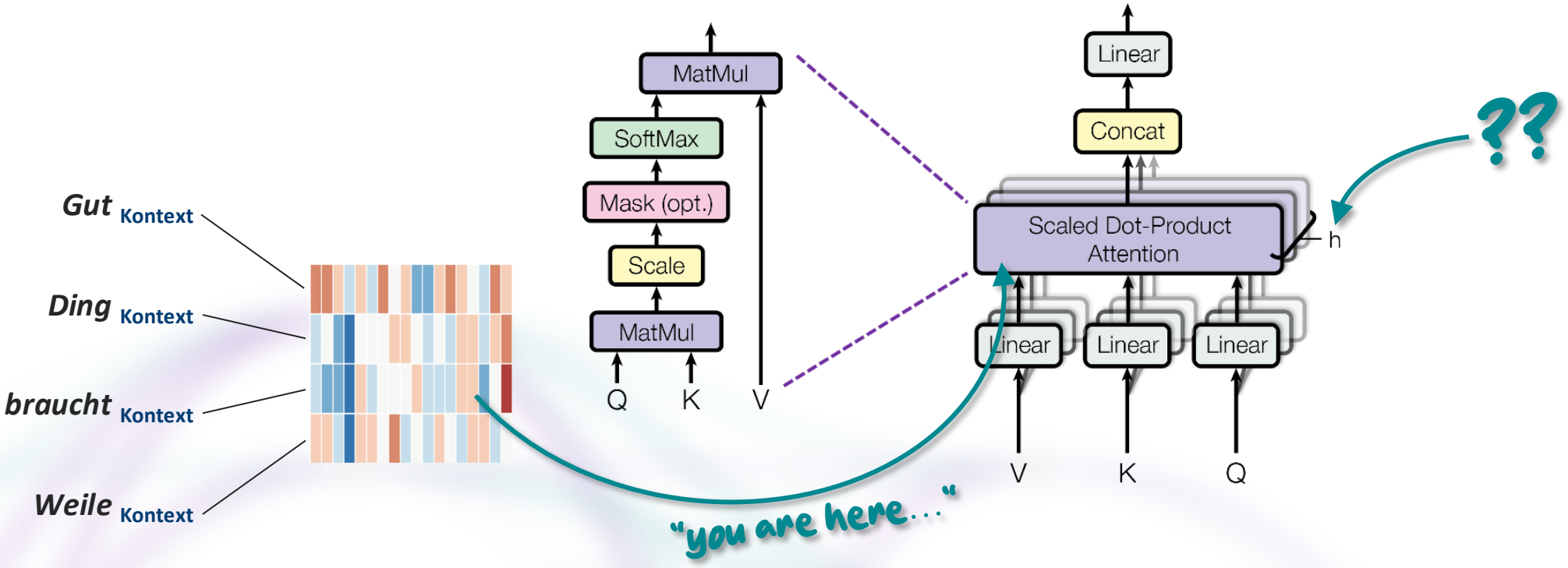
$$M_{ij} = \begin{cases} 0 & j \le i \\ -\infty & j > i \end{cases}$$

$e^{-\infty} = 0$: Gewicht exakt 0, Zeilensumme trotzdem 1

- **Training:** Ohne Maske könnte Position i das Ziel (Token i+1) einfach ablesen – das Modell würde abschreiben statt vorhersagen.
- **Inferenz:** Spätere Tokens gibt es noch nicht; die Maske sorgt dafür, dass das Modell genauso rechnet wie im Training.
- **Zahlen:** dieselben Scores wie zuvor; die erlaubten Einträge jeder Zeile werden neu auf Summe 1 normiert. Die markierten Bezüge zeigen auf frühere Tokens und bleiben erhalten.
- Encoder (z. B. BERT, Übersetzungs-Encoder) rechnen ohne kausale Maske.

Vaswani et al. (2017), Abschn. 3.2.3: „masking out (setting to $-\infty$) all values in the input of the softmax which correspond to illegal connections“ · Devlin et al. (2019): BERT „uses bidirectional self-attention“.

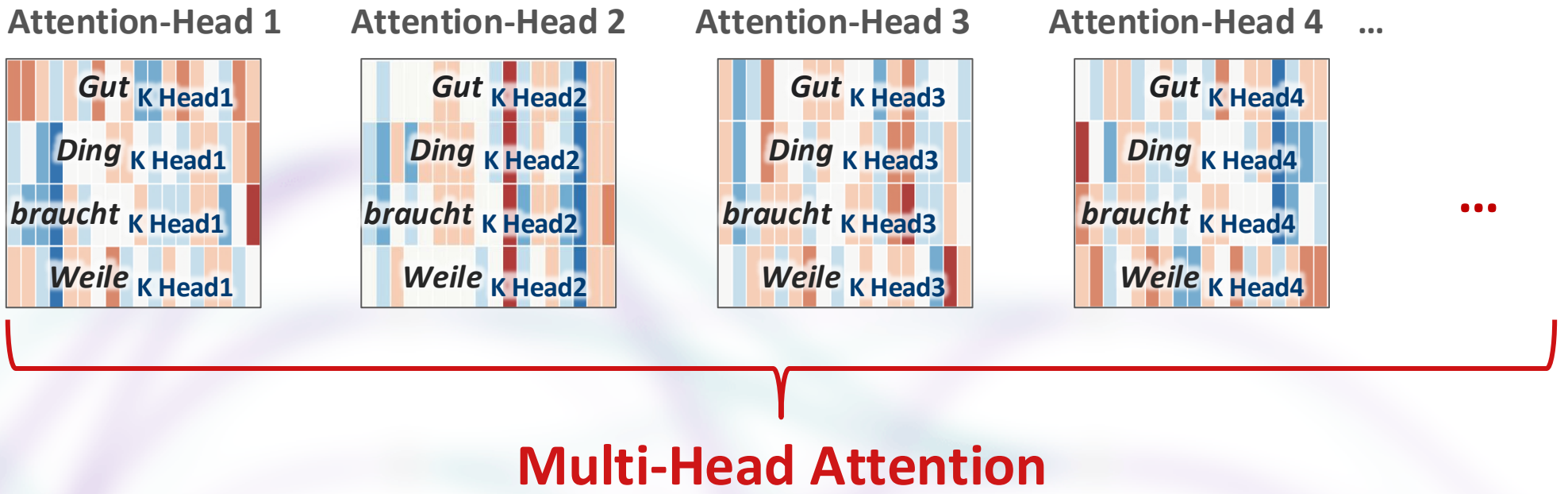
3. Attention | Ein Attention-Kopf (Head)



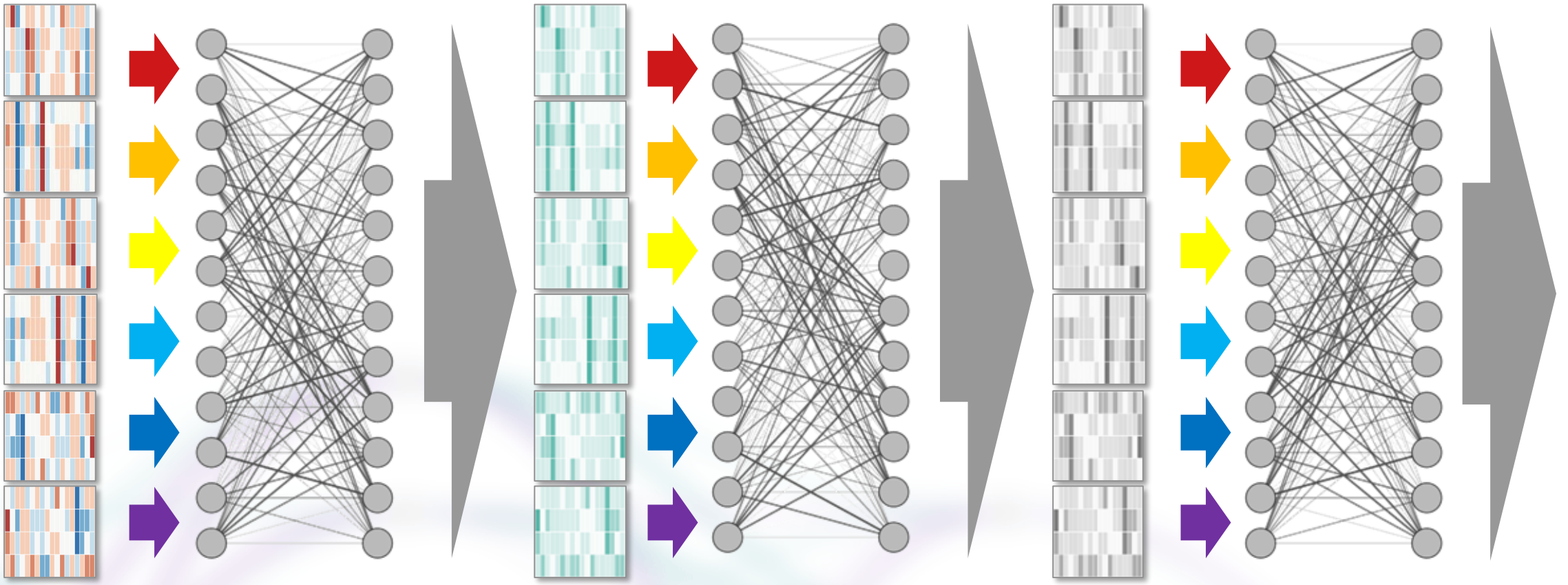
VASWANI, Ashish, et al. Attention is all you need. Advances in neural information processing systems, 2017, 30. Jg.

3. Attention | Mehrere Köpfe: Multi-Head-Attention

- Mehrere Köpfe rechnen parallel, jeder mit eigenen W_Q, W_K, W_V im kleineren Raum $d_k = d_{model} / h$.
- Spezialisierung auf verschiedene Beziehungen ist eine Beobachtung, keine Vorgabe –
- viele Köpfe lassen sich nachträglich entfernen (Michel et al. 2019).



3. Attention | Köpfe zusammenführen: Concat und W_O



- Jeder Kopf: 1/6 von d_{model} (Guppy: 6 × 64 = 384).
- Ausgaben verketten, dann mit W_O auf d_{model} projizieren.

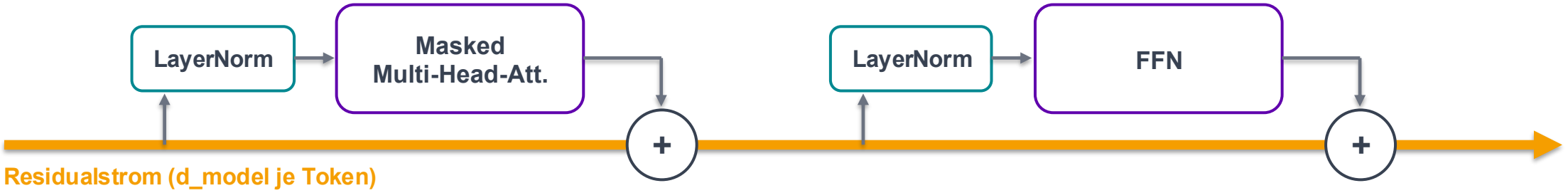
$$\mathrm{MHA}(X) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_h)\, W_O$$

$$\mathrm{head}_i = \mathrm{Attention}\left(XW_Q^i,\, XW_K^i,\, XW_V^i\right)$$



4. Transformer-Block | Residualstrom: addieren statt ersetzen

Jedes Teilmodul liest aus dem Strom und addiert seinen Beitrag (Pre-LN, wie GPT-2 und Guppy):



$$\mathbf{h} = \mathbf{x} + \mathrm{MHA}\big(\mathrm{LN}(\mathbf{x})\big)$$

$$\mathbf{x}' = \mathbf{h} + \mathrm{FFN}\big(\mathrm{LN}(\mathbf{h})\big)$$

- Attention tauscht Information zwischen Positionen aus; das FFN arbeitet je Position.
- Guppy-Beobachtung: Ein untrainierter Block ändert den Strom nur um 28,7 % – er ergänzt, statt zu ersetzen.

- Direkte Additionspfade erleichtern das Training sehr tiefer Netze (He et al. 2016).
- Ohne Skip-Verbindungen und FFN konvergiert reine Attention mit der Tiefe doppelt exponentiell gegen eine Rang-1-Matrix – alle Tokens würden gleich (Dong et al. 2021).
- Bild „Kommunikationskanal“: Jede Schicht liest aus dem Strom und schreibt hinein (Elhage et al. 2021).

He et al. (2016), CVPR · Dong, Cordonnier & Loukas (2021), ICML · Elhage et al. (2021), transformer-circuits.pub · Original 2017: Post-LN, LayerNorm(x + Sublayer(x))



4. Transformer-Block | LayerNorm: Pre-LN statt Post-LN

$$\mathrm{LN}(\mathbf{x}) = \boldsymbol{\gamma} \odot \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \boldsymbol{\beta}$$

$$\mu = \frac{1}{d}\sum_{k=1}^{d} x_k\,, \qquad \sigma^2 = \frac{1}{d}\sum_{k=1}^{d}(x_k - \mu)^2$$

Normiert wird jeder Token-Vektor für sich über seine d_{model} Komponenten; γ und β sind gelernte Parameter (Ba et al. 2016).

	Formel	Verwendet in
Post-LN	x ← LN(x + Sublayer(x))	Original-Transformer 2017, GPT-1
Pre-LN	x ← x + Sublayer(LN(x)), am Ende eine finale LN	GPT-2, GPT-3, GuppyLM

- **Post-LN braucht eine sorgfältige Lernraten-Aufwärmphase; Pre-LN hat bei der Initialisierung gutartige Gradienten (Xiong et al. 2020).**
- **Guppy nutzt trotzdem 200 Warmup-Schritte (Kap. 6).**

Ba, Kiros & Hinton (2016) · Xiong et al. (2020), ICML · Brown et al. (2020), Abschn. 2.1: GPT-3 nutzt die GPT-2-Architektur „including the modified initialization, pre-normalization“



4. Transformer-Block | Das FFN: je Position gleich, je Block verschieden

$$\mathrm{FFN}(\mathbf{x}) = \varphi(\mathbf{x}W_1 + \mathbf{b}_1)\,W_2 + \mathbf{b}_2$$

- **φ: ReLU (2017) bzw. GELU (GPT, Guppy). Ohne φ wären W_1 und W_2 zusammen nur eine lineare Abbildung.**
- **Breite $d_{ff} = 4 \cdot d_{model}$ (2017: 2 048; Guppy: 1 536).**
- **Dieselben Gewichte für alle Positionen eines Blocks – aber jeder Block hat sein eigenes FFN: „they use different parameters from layer to layer" (Vaswani et al. 2017).**
- **Parameter je Block: Attention $4d^2 + 4d$, FFN $8d^2 + 5d$ → FFN ≈ 2/3. Guppy: 1 181 568 von 1 774 464 = 66,6 %.**

Guppy: 6 Blöcke, jeder mit eigenem Satz

Block	Attention	FFN	Norm
1	$W_Q^1\ W_K^1\ W_V^1\ W_O^1$	$W_1^1\ W_2^1$	γ^1, β^1
2	$W_Q^2\ W_K^2\ W_V^2\ W_O^2$	$W_1^2\ W_2^2$	γ^2, β^2
3	$W_Q^3\ W_K^3\ W_V^3\ W_O^3$	$W_1^3\ W_2^3$	γ^3, β^3
…	…	…	…
6	$W_Q^6\ W_K^6\ W_V^6\ W_O^6$	$W_1^6\ W_2^6$	γ^6, β^6

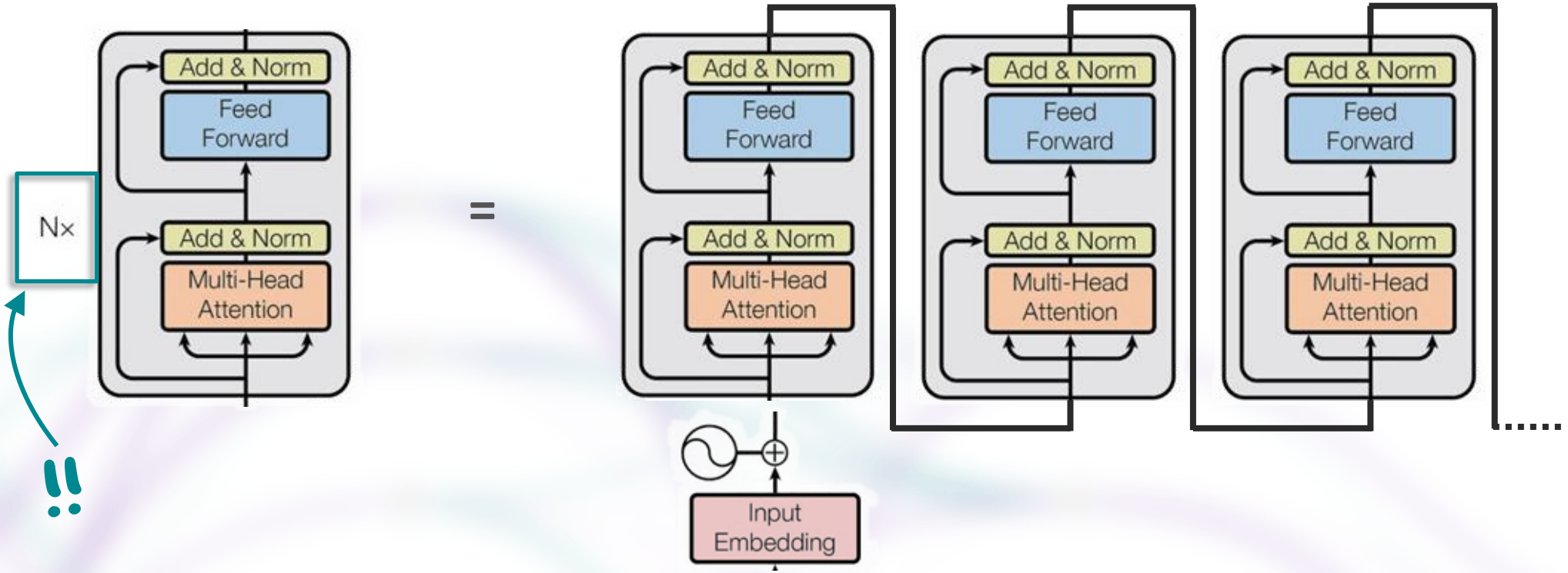
Keine Gewichte werden zwischen Blöcken geteilt. Nur die Embedding-Matrix wird am Ausgang wiederverwendet (Weight Tying).

Vaswani et al. (2017), Abschn. 3.3 · Radford et al. (2018): GELU · Geva et al. (2021): „Feed-forward layers constitute two-thirds of a transformer model's parameters" · Guppy: Notebook-Ausgabe



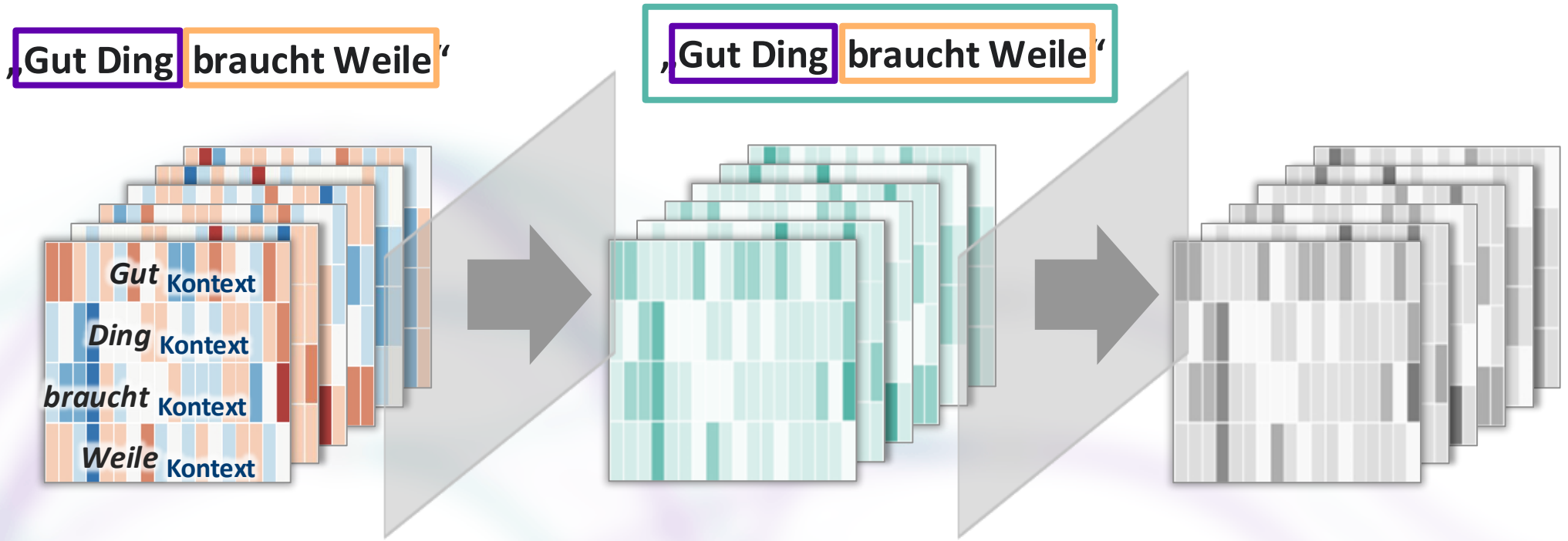
4. Transformer-Block | N Blöcke hintereinander

- Hintereinander geschaltet werden ganze Blöcke (Attention + FFN + Add & Norm) mit jeweils eigenen Gewichten – Guppy: 6, GPT-3: 96. Schon eine Attention-Schicht verbindet alle Positionen; mehr Tiefe erlaubt Komposition (z. B. Induction Heads, Elhage et al. 2021).



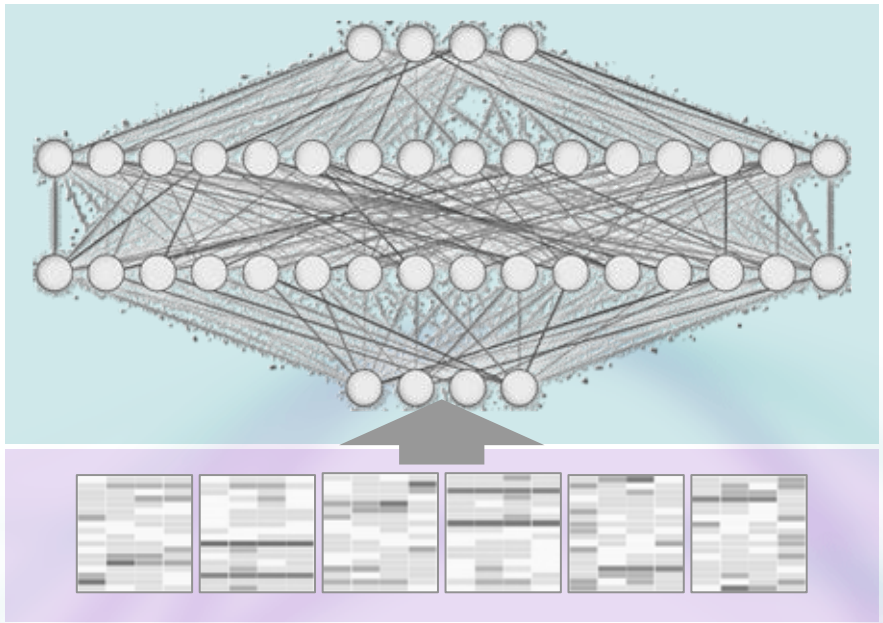
4. Transformer-Block | N Blöcke hintereinander

- Von Block zu Block wird die Repräsentation jedes Tokens verfeinert: Spätere Blöcke arbeiten mit Vektoren, in die schon Kontext eingemischt ist. Welche „Ebene“ (Satzteil, Satz, Absatz) ein Block erfasst, ist nicht vorgegeben, sondern entsteht im Training.



4. Transformer-Block | Das FFN als Wissensspeicher?

Bills et al. (2023): Language models can explain neurons in language models (OpenAI)



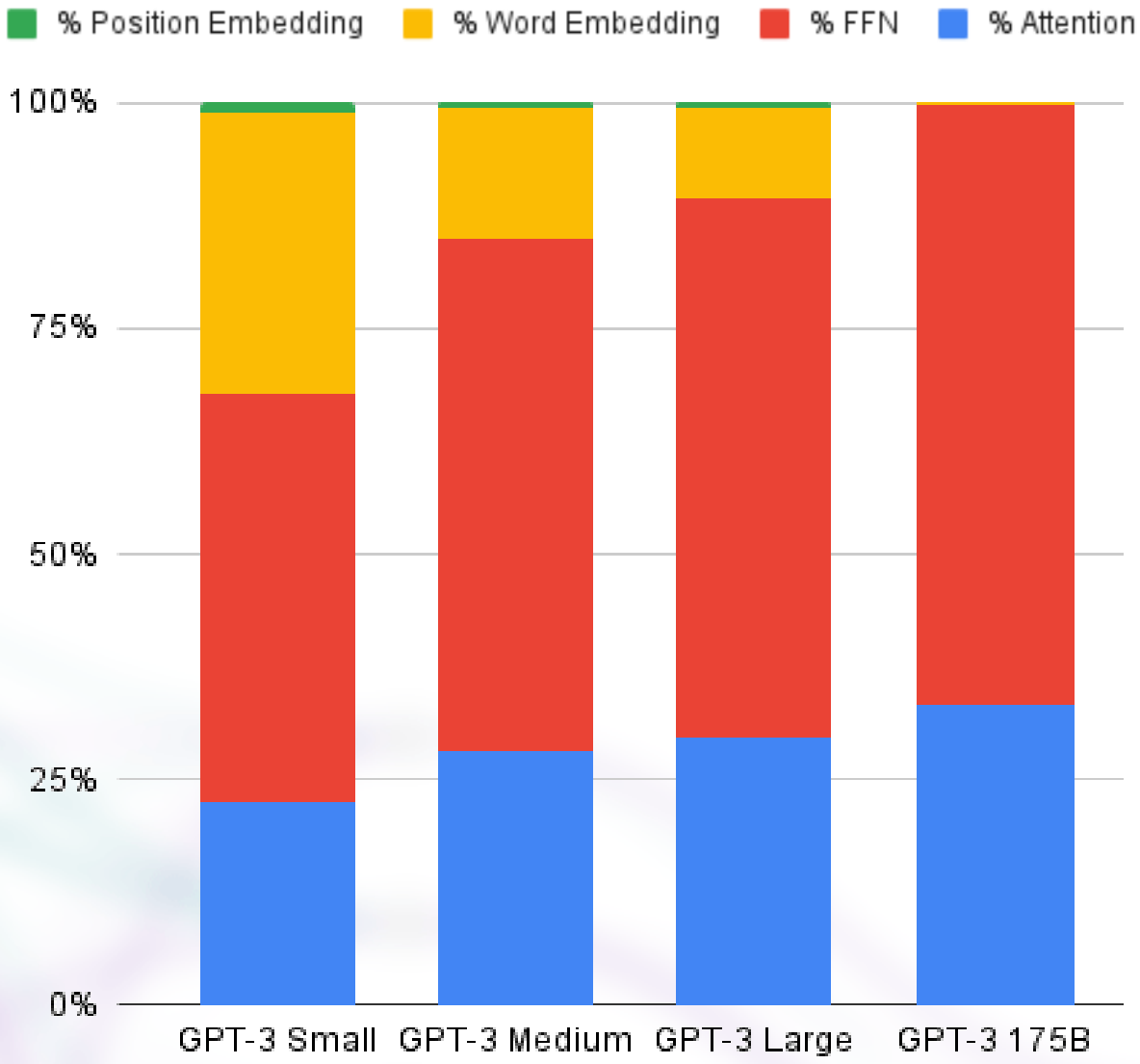
- FFN-Schichten enthalten rund 2/3 der Block-Parameter.
- Befund: FFNs wirken wie Schlüssel-Wert-Speicher; untere Schichten erfassen eher Oberflächenmuster, obere eher semantische (Geva et al. 2021).
- Faktenassoziationen lassen sich in mittleren FFN-Schichten lokalisieren und gezielt editieren (Meng et al. 2022).
- Aber: Die Lokalisierung sagt kaum voraus, in welcher Schicht Editieren am besten wirkt (Hase et al. 2023).

4. Transformer-Block | Aufteilung der Parameter (GPT-3)

Im GPT-3 175B Modell:

- ≈ 2/3 FFN
- ≈ 1/3 Attention
- < 1 %: Token- und Positions-Embeddings

- **GPT-4: Architektur nicht veröffentlicht (OpenAI 2023)**



[1] https://www.lesswrong.com/posts/3duR8CrvcHywrnhLo/how-does-gpt-3-spend-its-175b-parameters



5. Ausgabe | Vom letzten Vektor zur Wahrscheinlichkeit

$$\ell_t = \mathrm{LN}\left(\mathbf{h}_t^{(N)}\right) W_E^{\top} \in \mathbb{R}^V$$

$$P(x_{t+1} = w \mid x_{\le t}) = \frac{\exp(\ell_{t,w})}{\sum_{w'} \exp\left(\ell_{t,w'}\right)}$$

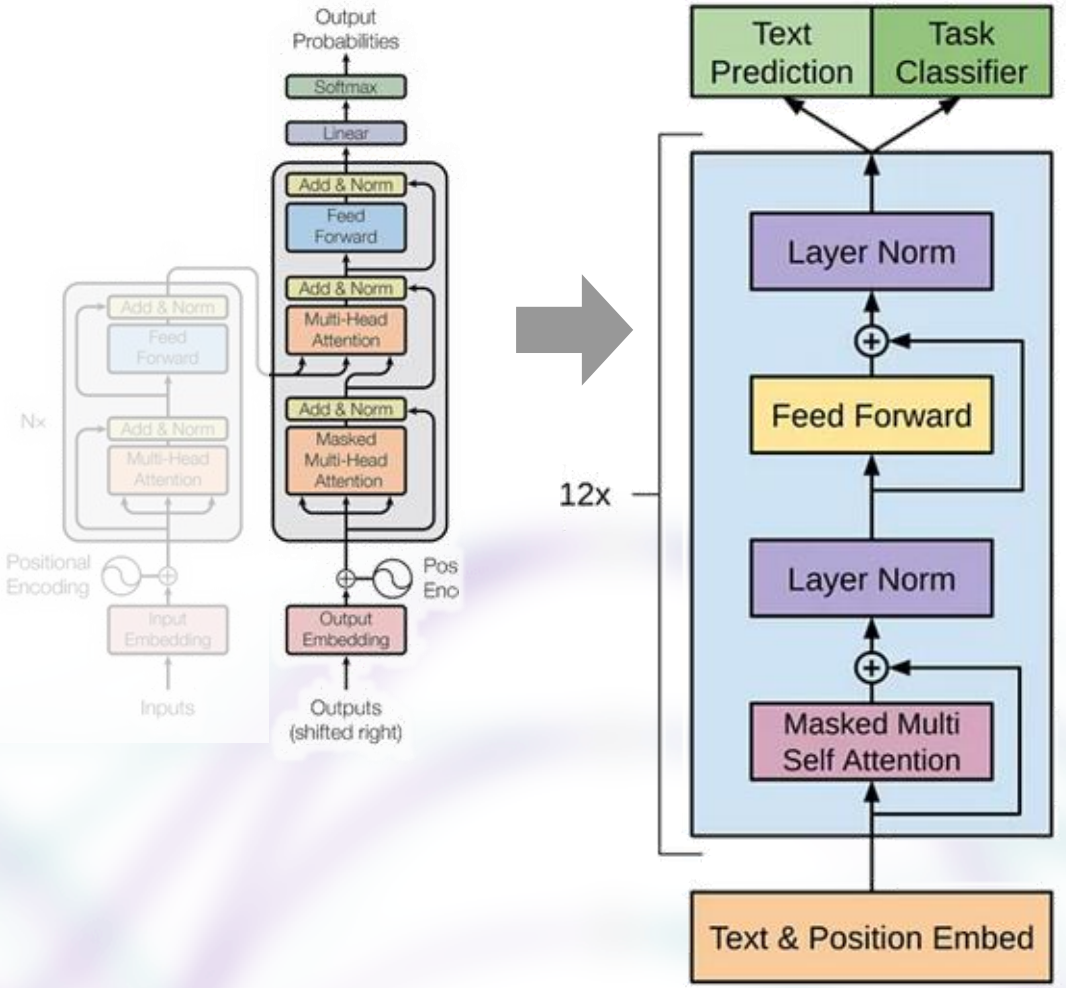
- ℓ_t: Logits – eine Punktzahl je Token im Vokabular
- $h_t^{(N)}$: Residualstrom an Position t nach dem letzten Block
- Softmax: Logits → Wahrscheinlichkeiten (Summe 1)

Das FFN schreibt – wie die Attention – einen d_{model}-Vektor in den Residualstrom.

- **Erst nach dem letzten Block: finale LayerNorm → LM-Head (lineare Abbildung d_model → V) → Softmax.**
- **Weight Tying: LM-Head = $W_E^{\top}$. Ein Logit ist also das Skalarprodukt aus Ausgangsvektor und Token-Embedding (Vaswani et al. 2017; Press & Wolf 2017). Guppy spart so 997 248 Parameter (8 %).**
- **An jeder Position entsteht eine Verteilung – im Training werden alle genutzt, bei der Inferenz nur die letzte.**

Vaswani et al. (2017), Abschn. 3.4: „we share the same weight matrix between the two embedding layers and the pre-softmax linear transformation“ · Press & Wolf (2017), EACL

5. Decoder-only | GPT, Gemini & Co.

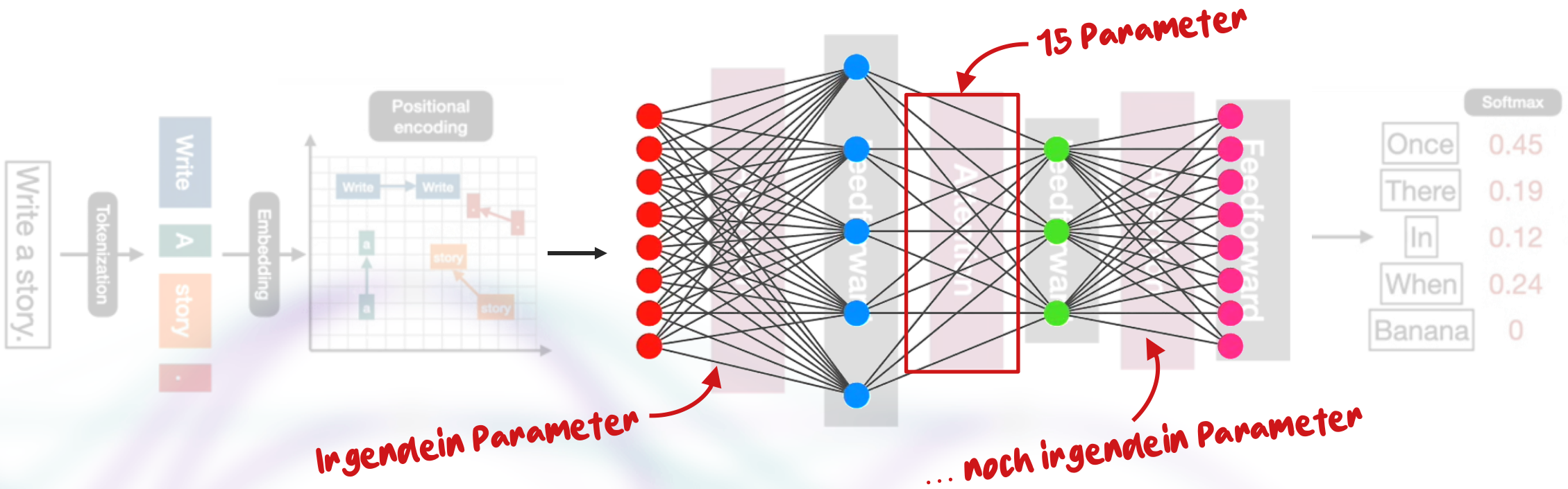


- GPT-Modelle (seit 2018) und Gemini nutzen nur einen Decoder-Stapel: ohne Encoder, ohne Cross-Attention.
- Prompt und Antwort laufen durch denselben Stapel – mit kausaler Maske.
- Ein Trainingsziel: das nächste Token (Kap. 6).
- Abb.: GPT-1 (2018) mit LayerNorm noch nach dem Teilmodul (Post-LN); ab GPT-2 davor (Pre-LN).

Radford et al. (2018): Improving Language Understanding by Generative Pre-Training. · Gemini Team (2023): Gemini – „build on top of Transformer decoders“.

6. Training: Was wird eigentlich gelernt?

- Parameter = alle Zahlen in allen Gewichtsmatrizen – das Training passt jede einzelne an.



What are Transformer Models and how do they work? (Luis Serrano)
https://www.youtube.com/watch?v=qaWMOYf4ri8



6. Training | Das Ziel: das nächste Token vorhersagen

Position t	0	1	2	3
Eingabe x	<s>	Gut	Ding	braucht
Modell sieht	<s>	<s> Gut	… Ding	… braucht
Ziel y	Gut	Ding	braucht	Weile

- y ist x um eine Position verschoben.
- Dank Maske: alle T Aufgaben in einem einzigen Durchlauf, parallel.
- Die Ziele stehen im Text selbst – selbstüberwachtes Lernen.

Verlust: Kreuzentropie

$$\mathcal{L}(\theta) = -\frac{1}{T}\sum_{t=1}^{T} \log P_\theta(x_{t+1} \mid x_{\leq t})$$

- Pro Position zählt nur, welche Wahrscheinlichkeit das Modell dem tatsächlich folgenden Token gibt.
- Referenz: Ein Modell, das alle V Tokens gleich wahrscheinlich hält, hat den Verlust ln V.
  Guppy untrainiert: 7,98 – ln 2 597 = 7,86.
- Padding-Positionen werden aus dem Verlust ausgeschlossen (Guppy: ignore_index).

Radford et al. (2018), Gl. 1: $L_1(U) = \Sigma_i \log P(u_i \mid u_{i-k}, \ldots, u_{i-1}; \Theta)$ · GuppyLM-Lehrnotebook, Abschn. 3 und 4.5

6. Training | Ein Trainingsschritt



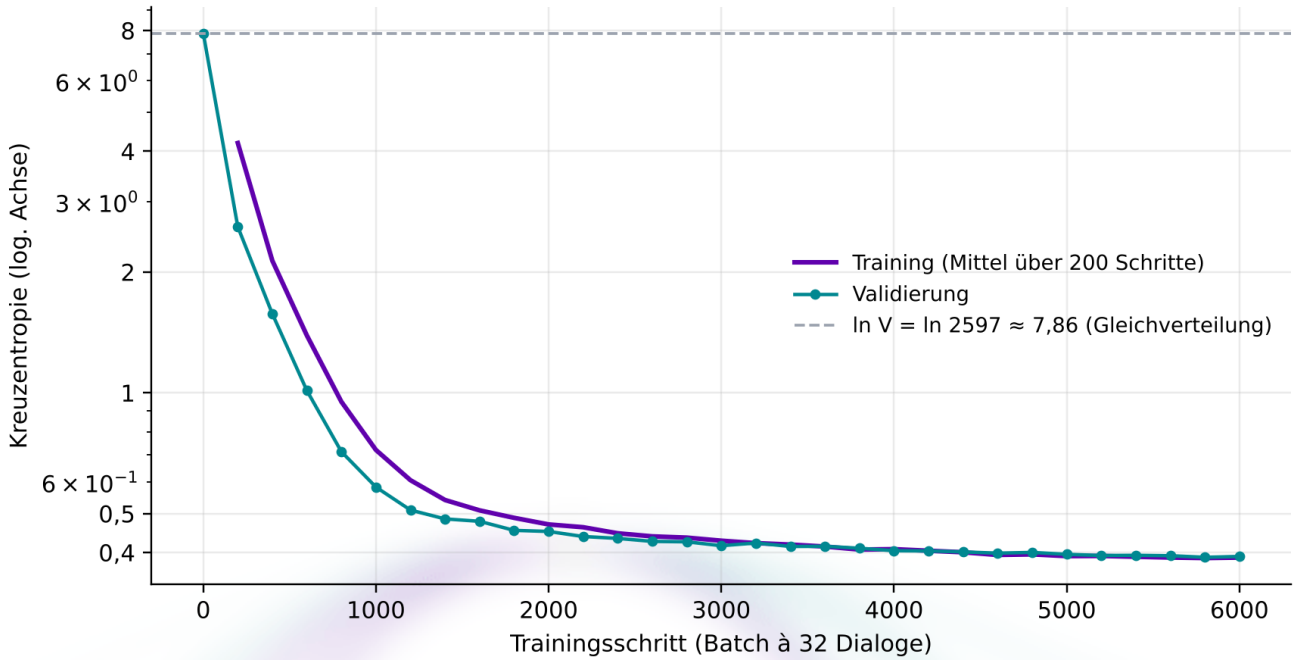
$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$$

Grundidee Gradientenabstieg; AdamW passt die Schrittweite je Parameter an und zieht Gewichte leicht Richtung 0 (Weight Decay).

- **Angepasst werden alle Parameter: W_E, W_Q, W_K, W_V, W_O, W_1, W_2, γ, β – in jedem Block eigene (eine gelernte Positionstabelle gegebenenfalls ebenso).**

Einstellung (Guppy)	Wert
Schritte × Batch	6 000 × 32
max. Lernrate η	$3 \cdot 10^{-4}$
Warmup	200 Schritte linear
danach	Kosinus bis $3 \cdot 10^{-5}$
Weight Decay	0,1
Gradient Clipping	Norm ≤ 1,0
Optimierer	AdamW, β = (0,9; 0,95)

6. Training | Die Verlustkurve von GuppyLM lesen



- **Start bei ≈ ln V: Das untrainierte Modell rät nahezu gleichverteilt.**
- **Nach 6 000 Schritten: bester Validierungsverlust 0,389 – entspricht einer Perplexität e^0,389 ≈ 1,48.**
- **Steigt die Validierung, während das Training sinkt, deutet das auf Überanpassung.**
- **Vorsicht: 2 320 der 3 000 Validierungsdialoge stehen wortgleich auch in den Trainingsdaten. Die Kurve belegt daher keine Generalisierung.**

Daten: Ausgabe der Trainingszelle im GuppyLM-Lehrnotebook (Validierung alle 200 Schritte auf 320 Dialogen); Hinweis zur Datenüberschneidung aus dem Notebook.



6. Training | Vom Sprachmodell zum Assistenten

Vortraining
nächstes Token vorhersagen auf sehr großen Textmengen

→

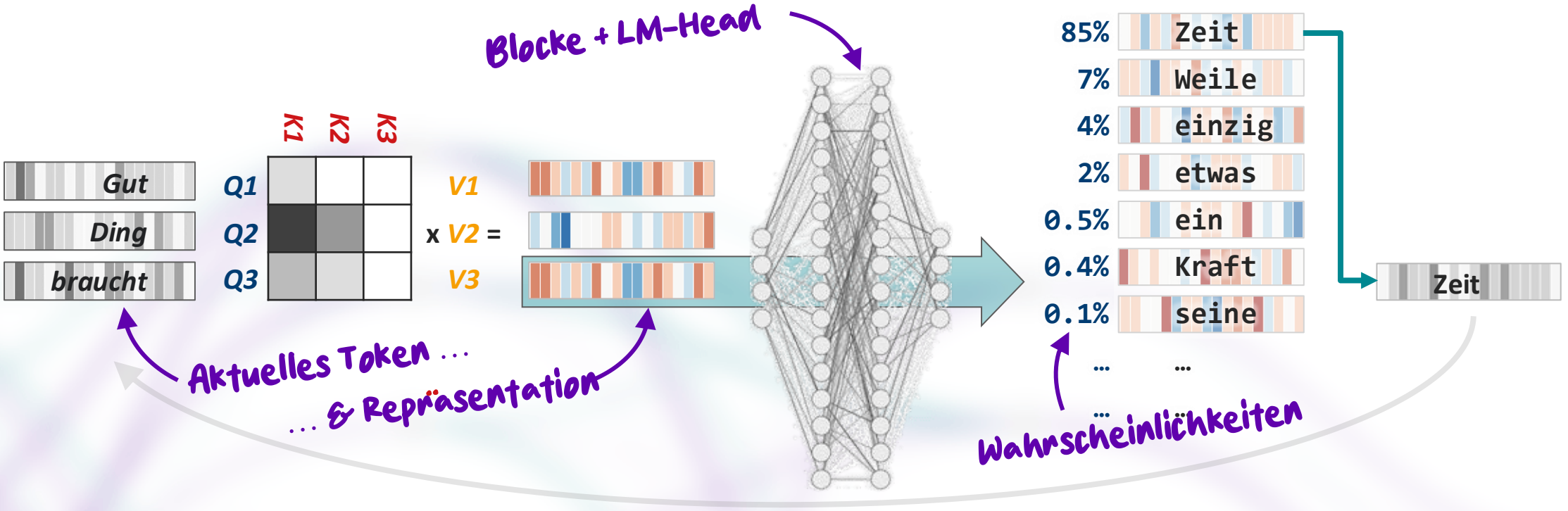
Feinabstimmung
auf Dialoge und mit menschlichem Feedback (RLHF)

- **GPT-4 ist laut OpenAI „a Transformer-style model pre-trained to predict the next token in a document“ und wurde anschließend mit Reinforcement Learning from Human Feedback feinabgestimmt.**
- **Die Architektur bleibt dabei gleich – es ändern sich Daten und Trainingsziel.**
- **GuppyLM überspringt das Vortraining: Es lernt direkt aus 57 000 kurzen Dialogen im Chat-Format `<|im_start|>user … <|im_end|>` – darum spricht es nur „Fisch“.**

7. Inferenz | Start der Textgenerierung

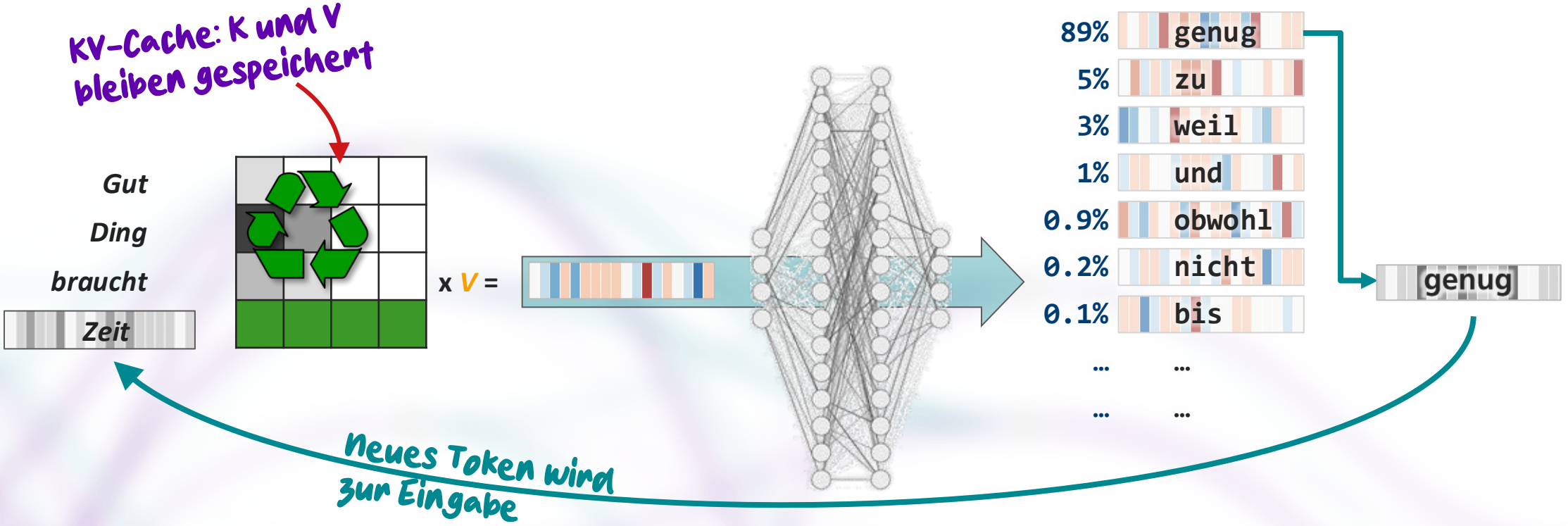
- Für die Vorhersage zählt nur der Ausgangsvektor der letzten Position.
- LayerNorm → LM-Head → Softmax machen daraus eine Verteilung über alle Tokens.
- Daraus wird das nächste Token gewählt (wahrscheinlichstes oder gezogen).

Ausprobieren: Lernapp „Transformer-Werkbank“



7. Inferenz | Autoregression und KV-Cache

- Die neue Tokenfolge ist um ein Token länger als die vorherige.
- Sie wird im nächsten Schritt wieder zur Eingabe des Modells.
- Dieses Prinzip wird als "Autoregression" bezeichnet.



KV-Cache: Für das neue Token wird nur seine Zeile berechnet; Keys und Values früherer Tokens werden wiederverwendet (Pope et al. 2022). GuppyLM rechnet ohne Cache jedes Mal neu.



7. Inferenz | Wie wird das nächste Token gewählt?

$$P_T(w) = \frac{\exp(\ell_w/\tau)}{\sum_{w'} \exp(\ell_{w'}/\tau)}$$

GuppyLM, Frage „hello guppy“, top-k 50

- **Temperatur τ = 0: immer das wahrscheinlichste Token (greedy).**
- **T < 1: Verteilung spitzer, vorhersagbarer; T > 1: flacher, abwechslungsreicher bis wirr.**
- **top-k: nur die k wahrscheinlichsten Tokens bleiben im Rennen.**
- **Stopp beim Endtoken, beim Tokenlimit oder an der Kontextgrenze.**

Bei T > 0 wird gezogen: Dieselbe Frage kann verschiedene Antworten liefern.

T	Antwort (Notebook-Ausgabe)
0	hi there. i just found a nice spot in my favorite spot.
0,7	hi there. did you bring the red flakes.
0,7	hi there. i just found a nice spot near the bottom.
1,5	hi there. did you bring freeze dried worms. the bubbles feels nice.

GuppyLM-Lehrnotebook, Abschn. 6 (Funktion erzeuge, Temperatur-Beobachtung)



8. Größenordnungen | GPT-3 im Vergleich

Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}
GPT-3 Small	125M	12	768	12	64
GPT-3 Medium	350M	24	1024	16	64
GPT-3 Large	760M	24	1536	16	96
GPT-3 175B or "GPT-3"	175.0B	96	12288	96	128

Trainingsdaten GPT-3: Common Crawl 45 TB komprimiert vor, 570 GB nach Filterung (plus weitere Quellen); trainiert auf 300 Mrd. Tokens [2]

Gewichte: 175 Mrd. Parameter × 2 Byte (fp16) ≈ 350 GB, in fp32 ≈ 700 GB (eigene Rechnung)

[1] https://www.lesswrong.com/posts/3duR8CrvcHywrnhLo/how-does-gpt-3-spend-its-175b-parameters

[2] https://arxiv.org/pdf/2005.14165.pdf

Zum Vergleich GuppyLM: 11,7 Mio. Parameter · 6 Blöcke · d_model = 384 · 6 Köpfe · ≈ 47 MB in fp32

8. Emergente Eigenschaften? | Ein umstrittener Begriff

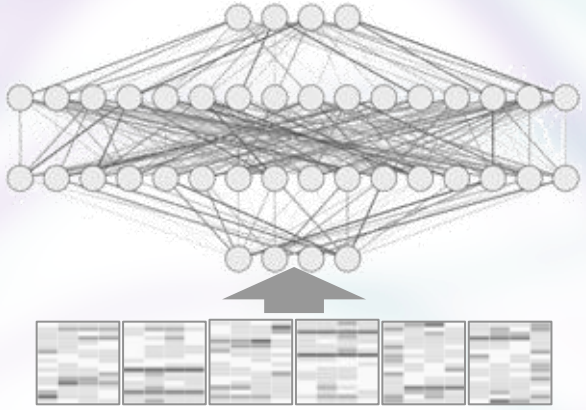
Bubeck et al. (2023): Sparks of Artificial General Intelligence, Abschn. 1 · Schaeffer et al. (2023): Are Emergent Abilities of LLMs a Mirage? NeurIPS

Emergenz (allgemein): Merkmale, die entstehen,

wenn einzelne Komponenten zu komplexen Systemen zusammenwirken,

welche neue, unvorhersehbare Verhaltensweisen aufweisen,

die nicht aus den Eigenschaften der einzelnen Komponenten abzuleiten sind.



Bubeck et al. (2023) über GPT-4: Es zeige „emergent behaviors and capabilities whose sources and mechanisms are, at this moment, hard to discern precisely“.

Gegenposition: Scheinbar sprunghafte Fähigkeiten können von der Wahl der Messgröße abhängen – mit einer anderen Metrik verschwindet der Sprung (Schaeffer et al. 2023).

Offen bleibt, was die Modelle intern tatsächlich berechnen – Gegenstand der Interpretierbarkeitsforschung.



Anhang: Formelsammlung | Schreibweisen und Operationen

Schreibweise	Bedeutung	Beispiel
x, q, k, v (fett, klein)	Vektor – hier immer als Zeilenvektor geschrieben	x = (0,2; −1,3; …)
W, Q, K, V (groß)	Matrix	W_Q ∈ ℝ^(384×64)
ℝ^(m×n), ∈	Menge der reellen Matrizen mit m Zeilen und n Spalten; „ist Element von“	W_E ∈ ℝ^(2597×384)
a_i, s_{ij}	Komponente i eines Vektors bzw. Eintrag in Zeile i, Spalte j	s_{23}: Query 2, Key 3
$x^{(b)}$	hochgestellt in Klammern = Index (hier Block b), kein Exponent	$x^{(0)}$: Eingabe des 1. Blocks
x′, x̃	Strich bzw. Tilde: veränderte Version von x	x′ = Ausgabe eines Blocks
A^T	Transponierte: Zeilen und Spalten vertauscht	(2×3-Matrix)^T ist 3×2
|a|	Länge (Betrag) eines Vektors: $\sqrt{(\Sigma\, a_i^2)}$	|(3, 4)| = 5
a · b, xW	Skalarprodukt zweier Vektoren bzw. Vektor-Matrix-Produkt	(1, 2)·(3, 4) = 11
a ⊙ b	Hadamard-Produkt: elementweise Multiplikation gleich langer Vektoren	(1, 2, 3) ⊙ (4, 5, 6) = (4, 10, 18)
Σ_j	Summe über alle Werte des Index j	$\Sigma_{j=1}^{3}\, j = 6$
Concat(…)	Verkettung: Vektoren aneinanderhängen	Concat((1, 2), (3)) = (1, 2, 3)
exp(z), log(z)	e^z bzw. natürlicher Logarithmus ln z; log(exp(z)) = z	exp(0) = 1
−∞	„minus unendlich“: exp(−∞) = 0 – so wirkt die Maske	$M_{ij} = -\infty$ für j > i
P(A | B)	bedingte Wahrscheinlichkeit: A, wenn B bekannt ist	P(„Weile“ | „Gut Ding braucht“)
←	Zuweisung: der linke Wert wird durch den rechten ersetzt	θ ← θ − η∇L
∇_θ L	Gradient: Vektor aller partiellen Ableitungen ∂L/∂θ – zeigt bergauf	Training geht in −∇-Richtung



Anhang: Formelsammlung | Größen und Symbole

Größen (Namen wie in Brown et al. 2020, Tab. 2.1)

Symbol	Bedeutung	Guppy
n_vocab	Vokabulargröße	2 597
d_model	Breite des Residualstroms	384
n_layers	Anzahl Blöcke (Abb. 2017: „N×“)	6
n_heads	Köpfe je Block	6
d_k	Breite je Kopf = d_model / n_heads	64
d_ff	innere Breite des FFN	1 536
n_ctx	Kontextfenster (max. Tokens)	128
T	Länge der aktuellen Tokenfolge	≤ 128
θ	alle trainierbaren Parameter	11,6 Mio.
η	Lernrate	max. $3 \cdot 10^{-4}$
τ	Temperatur beim Sampling	0,7

Bezeichner in den Formeln

Symbol	Bedeutung
t, pos	Position eines Tokens (0, 1, 2, …)
i, j	Query-Position bzw. Key-Position
m, h, b	Frequenz-Index (Sinus), Kopf-Index, Block-Index
tok_t, w	Token-ID an Position t; beliebiges Token
W_E, p_t	Embedding-Matrix; Positionsvektor
W_Q, W_K, W_V, W_O	Projektionen der Attention
W_1, b_1, W_2, b_2	Gewichte und Bias-Vektoren des FFN
s_{ij}, α_{ij}, z_i	Score, Gewicht, Ausgabe der Attention
M	kausale Maske (0 oder $-\infty$)
μ, σ^2, ε	Mittelwert, Varianz, kleine Konstante (LN)
γ, β	gelernte Skalierung und Verschiebung (LN)
φ	Aktivierungsfunktion (ReLU, GELU)
ℓ_t, $\mathcal{L}$	Logits an Position t; Verlust

Guppy-Werte: Notebook-Ausgaben (θ: 11 644 800 trainierbar; τ = 0,7 ist der Standardwert von antworte).



Anhang: Formelsammlung | Die Formeln des Vortrags (1/2)

$\mathbf{a}\cdot\mathbf{b}=\sum_i a_i\,b_i=\lvert\mathbf{a}\rvert\,\lvert\mathbf{b}\rvert\cos\theta$	Skalarprodukt; θ ist der Winkel zwischen a und b.	→ F. 7
$\cos\theta=\frac{\mathbf{a}\cdot\mathbf{b}}{\lvert\mathbf{a}\rvert\,\lvert\mathbf{b}\rvert}$	Kosinusähnlichkeit: −1 … 1, unabhängig von den Längen.	→ F. 7
$\mathbf{x}_t^{(0)}=W_E[\mathrm{tok}_t]+\mathbf{p}_t$	Eingabe an Position t: Zeile tok_t der Embedding-Matrix plus Positionsvektor.	→ F. 15
$PE_{(pos,2m)}=\sin\left(\frac{pos}{10000^{2m/d_{\mathrm{model}}}}\right)$	Sinus-Positionskodierung; Dimension 2m+1 analog mit cos. m nummeriert die Frequenzen.	→ F. 19
$\mathbf{q}=\mathbf{x}W_Q,\ \mathbf{k}=\mathbf{x}W_K,\ \mathbf{v}=\mathbf{x}W_V$	Lineare Projektionen eines Eingabevektors (Zeilenvektor × Matrix).	→ F. 27
$s_{ij}=\frac{\mathbf{q}_i\cdot\mathbf{k}_j}{\sqrt{d_k}},\quad \alpha_{ij}=\frac{\exp(s_{ij})}{\sum_{j'}\exp(s_{ij'})}$	Score (Passung von Query i und Key j) und Gewicht (Softmax über Zeile i, Summe 1).	→ F. 30
$\mathbf{z}_i=\sum_j \alpha_{ij}\,\mathbf{v}_j$	Ausgabe eines Kopfes an Position i: gewichtete Summe der Values.	→ F. 30
$\mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}+M\right)V$	Dasselbe in Matrixform für alle Positionen; M setzt spätere Positionen auf −∞.	→ F. 30, 33



Anhang: Formelsammlung | Die Formeln des Vortrags (2/2)

$$\mathrm{MHA}(X) = \mathrm{Concat}\big(\mathrm{head}_1, \ldots, \mathrm{head}_{n_{\mathrm{heads}}}\big)\, W_O$$	Köpfe verketten und mit W_O auf d_model zurückprojizieren.	→ F. 36
$$\tilde{\mathbf{x}} = \mathbf{x} + \mathrm{MHA}\big(\mathrm{LN}(\mathbf{x})\big), \quad \mathbf{x}' = \tilde{\mathbf{x}} + \mathrm{FFN}\big(\mathrm{LN}(\tilde{\mathbf{x}})\big)$$	Ein Block (Pre-LN): beide Teilmodule addieren ihren Beitrag zum Residualstrom.	→ F. 37
$$\mathrm{LN}(\mathbf{x}) = \boldsymbol{\gamma} \odot \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \boldsymbol{\beta}$$	LayerNorm je Token; ⊙ = elementweise Multiplikation.	→ F. 38
$$\mathrm{FFN}(\mathbf{x}) = \varphi(\mathbf{x}W_1 + \mathbf{b}_1)W_2 + \mathbf{b}_2$$	φ: ReLU(z) = max(0, z) oder GELU(z) = z·Φ(z), Φ = Verteilungsfunktion der Standardnormalverteilung.	→ F. 39
$$\ell_t = \mathrm{LN}\left(\mathbf{x}_t^{(n_{\mathrm{layers}})}\right) W_E^{\top}, \quad P(\mathrm{tok}_{t+1} = w \mid \mathrm{tok}_{\leq t}) = \mathrm{softmax}(\ell_t)_w$$	Logits und Wahrscheinlichkeit des nächsten Tokens.	→ F. 44
$$\mathcal{L}(\theta) = -\frac{1}{T}\sum_t \log P_\theta(\mathrm{tok}_{t+1} \mid \mathrm{tok}_{\leq t})$$	Kreuzentropie-Verlust; Perplexität = exp($\mathcal{L}$).	→ F. 47, 49
$$\theta \leftarrow \theta - \eta\, \nabla_\theta \mathcal{L}$$	Gradientenabstieg: Parameter ein Stück gegen den Gradienten verschieben.	→ F. 48
$$P_\tau(w) = \frac{\exp(\ell_w/\tau)}{\sum_{w'} \exp(\ell_{w'}/\tau)}$$	Sampling mit Temperatur τ.	→ F. 53

GELU: Hendrycks & Gimpel (2016). Folienverweise beziehen sich auf diesen Foliensatz.



Literatur | Quellen (1/2)

Ba, J. L.; Kiros, J. R.; Hinton, G. E. (2016): Layer Normalization. arXiv:1607.06450

Bills, S. et al. (2023): Language models can explain neurons in language models. OpenAI

Brown, T. et al. (2020): Language Models are Few-Shot Learners. NeurIPS. arXiv:2005.14165

Bubeck, S. et al. (2023): Sparks of Artificial General Intelligence. arXiv:2303.12712

Devlin, J. et al. (2019): BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL

Dong, Y.; Cordonnier, J.-B.; Loukas, A. (2021): Attention is not all you need. ICML

Elhage, N. et al. (2021): A Mathematical Framework for Transformer Circuits. transformer-circuits.pub

Ethayarajh, K. (2019): How Contextual are Contextualized Word Representations? EMNLP

Firth, J. R. (1957): A synopsis of linguistic theory 1930–1955. Studies in Linguistic Analysis

Gao, J. et al. (2019): Representation Degeneration Problem in Training NLG Models. ICLR

Gemini Team (2023): Gemini: A Family of Highly Capable Multimodal Models. arXiv:2312.11805

Geva, M. et al. (2021): Transformer Feed-Forward Layers Are Key-Value Memories. EMNLP

Harris, Z. S. (1954): Distributional structure. Word 10 (2–3), 146–162

Hase, P. et al. (2023): Does Localization Inform Editing? NeurIPS



Literatur | Quellen (2/2)

Haviv, A. et al. (2022): Transformer LMs without Positional Encodings Still Learn Positional Information. Findings of EMNLP
He, K. et al. (2016): Deep Residual Learning for Image Recognition. CVPR
Loshchilov, I.; Hutter, F. (2019): Decoupled Weight Decay Regularization. ICLR
Meng, K. et al. (2022): Locating and Editing Factual Associations in GPT. NeurIPS
Michel, P.; Levy, O.; Neubig, G. (2019): Are Sixteen Heads Really Better than One? NeurIPS
Mikolov, T. et al. (2013): Efficient Estimation of Word Representations in Vector Space. arXiv:1301.3781
OpenAI (2023): GPT-4 Technical Report. arXiv:2303.08774
Pope, R. et al. (2022): Efficiently Scaling Transformer Inference. arXiv:2211.05102
Press, O.; Wolf, L. (2017): Using the Output Embedding to Improve Language Models. EACL
Radford, A. et al. (2018): Improving Language Understanding by Generative Pre-Training. OpenAI
Radford, A. et al. (2019): Language Models are Unsupervised Multitask Learners. OpenAI
Schaeffer, R.; Miranda, B.; Koyejo, S. (2023): Are Emergent Abilities of LLMs a Mirage? NeurIPS
Sennrich, R.; Haddow, B.; Birch, A. (2016): Neural Machine Translation of Rare Words with Subword Units. ACL
Vaswani, A. et al. (2017): Attention Is All You Need. NeurIPS. arXiv:1706.03762
Xiong, R. et al. (2020): On Layer Normalization in the Transformer Architecture. ICML
GuppyLM-Lehrnotebook (iludis), Bearbeitung von arman-bd/guppylm (MIT)